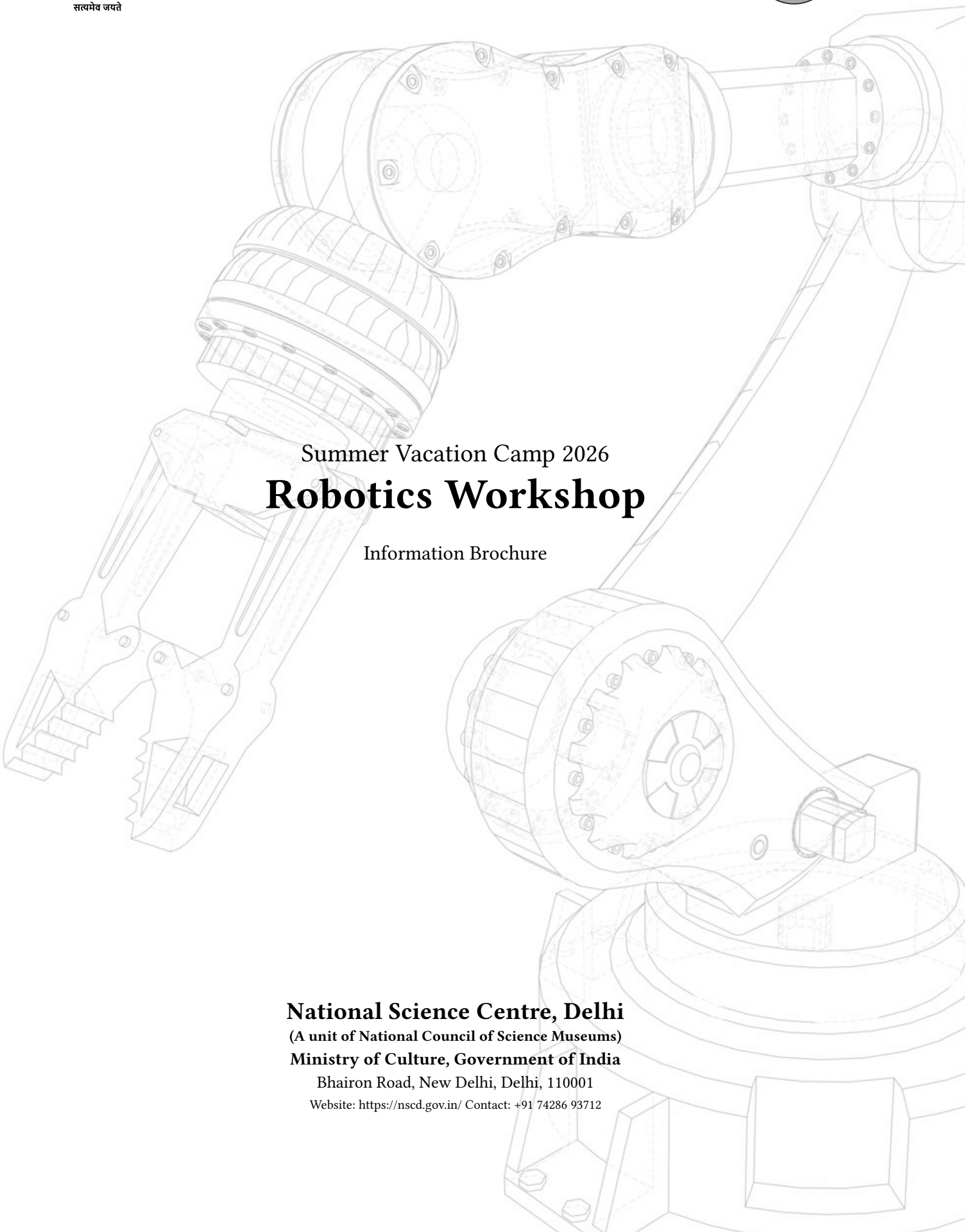
संस्कृति मंत्रालय
MINISTRY OF
CULTURE

सत्यमेव जयते



Summer Vacation Camp 2026

Robotics Workshop

Information Brochure

National Science Centre, Delhi

(A unit of National Council of Science Museums)

Ministry of Culture, Government of India

Bhairon Road, New Delhi, Delhi, 110001

Website: https://nscd.gov.in/ Contact: +91 74286 93712

01 June 2026 | v1.3

Workshop Schedule

This five-day workshop is designed to take you from your very first encounter with robotics all the way to building, programming, and competing with a real robotic arm. Each day builds on the one before it — theory first, then hands-on practice, then integration, then competition. Come prepared, stay curious, and make the most of every session.

Day 1

Introduction to Robotics

An exciting first look at the world of robotics — covering the history of robots, their anatomy and core components, real-world applications across industry, medicine, and space, and live demonstrations of robotic systems in action. By the end of the day you will understand what a robot is, how it is built, and why it matters.

Day 2

Mechanical Design and Assembly

A deep dive into the mechanical side of robotics — links, joints, degrees of freedom, gears, and fasteners. You will then apply this knowledge directly by assembling the mechanical structure of the robotic arm using 3D-printed parts and servo motors, step by step.

Day 3

Electronics Integration and Testing

The arm comes to life electrically. You will wire up the Arduino Nano, PCA9685 servo driver, and all four servo motors, connect the external power supply, and run diagnostic tests to verify every joint is responding correctly. Debugging and systematic fault-finding are key skills practised on this day.

Day 4

Programming and System Integration

With hardware fully assembled and tested, you will write and upload C/C++ programs using the Arduino framework — controlling servo positions through PWM, handling serial communication, and running the complete robot controller that brings all four joints under unified software control.

Day 5

Competition Day — *Moving Blocks*

The final day is yours to prove what you have built and learned. Teams go head to head in the Moving Blocks competition — a timed challenge where each robotic arm must move as many blocks as possible from one zone to another with precision and speed. Preliminary rounds lead to final rounds, and the best arm on the day takes the title.

Table of Contents

1 Welcome to the World of Robotics

Have you ever watched a robotic arm sweep across a car factory floor — welding metal, lifting heavy parts, and painting surfaces with perfect consistency — never tiring, never losing focus? Have you seen footage of a robot arm on the International Space Station, delicately repositioning equipment in the silence of space? Or perhaps you have heard about surgical robots that help doctors make incisions so precise that patients recover in days instead of weeks?

These are not scenes from a science fiction film. They are real, and the science that makes them possible is called **Robotics**.

Robotics is a captivating branch of science and engineering dedicated to designing and building machines that can perform tasks on their own — or with very little human guidance. These machines are called **robots**. What makes a robot remarkable is its ability to **sense** the world around it, **process** that information, and then **act** in a meaningful way. A robot is not just a mechanical gadget — it is an intelligent, integrated system.

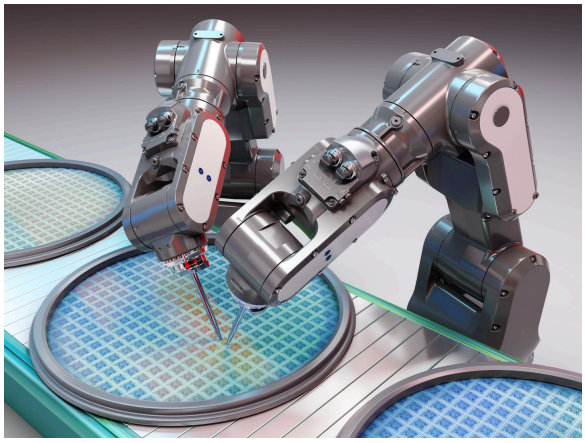


Figure 2: Robotic Arm used in Semiconductor Manufacturing Industry.

Robots come in an astonishing variety of shapes and sizes. Some are as large as a room; others are smaller than your thumb. Some roll on wheels, some walk on legs, others fly through the air or swim underwater. But almost all of them share the same fundamental set of ideas at their core — ideas that you are about to explore and understand.

1.1 Why Robotics Matters

Think about some of the most dangerous jobs in the world — cleaning sewers deep underground, fighting fires in collapsing buildings, handling radioactive materials in nuclear facilities, or disarming explosive devices. In every one of these situations, a human life is at risk. Now, imagine replacing the human with a robot. It does not feel pain. It does not get tired. It does not have a family waiting at home. It can go where humans cannot — and come back out with the job done.

This is one of the most compelling reasons why robotics matters: **it protects human lives**.

But the importance of robotics goes much further than safety. Robots help surgeons perform operations on parts of the body too delicate for the human hand. They help farmers harvest crops more efficiently, reducing food waste. They build cars, assemble electronics, and package products with a speed and accuracy no human workforce could match. In scientific research, robots handle chemicals and biological samples that would be hazardous to human researchers.

And now, with the rise of Artificial Intelligence, robots are becoming smarter — capable of learning, adapting, and making decisions in complex, unpredictable environments.

Figure 3: (Futuristic Image of) Students collaborating on robotics projects.

Here is the truly exciting part: **you can be part of this**. Learning robotics means learning to think like an inventor — to look at a real problem in the world and ask, "How could I build something to solve this?" You will combine science, electronics, coding, and creativity. You will develop skills that stretch far beyond any single classroom subject. Robotics is not just a field of engineering — it is a way of thinking that can change the world.

1.2 The Robotic Arm: A Perfect Learning Machine

There are many kinds of robots to explore, so why do we focus on the **robotic arm**?

Because a robotic arm is, quite simply, one of the most complete and educational systems in all of robotics. Unlike specialised robots built for just one narrow purpose, a robotic arm combines almost every fundamental concept of robotics into a single, elegant machine. Study a robotic arm deeply and you will understand robots broadly.

A robotic arm also has the advantage of being deeply **familiar**. It works in a way that mirrors your own arm — rigid segments linked together at moving joints, with a tool at the end to interact with the world. This makes it easier to reason about: you already know intuitively how an arm should move, even before you learn the engineering behind it.

While exploring a robotic arm, you will encounter and understand mechanical movement and joints, motors and electronics, sensors and control systems, programming and automation, and the thinking that goes into precision design.

Figure 4: An open-source DIY robotic arm. (AR4 by Articulated Robotics)

Workshop: In this workshop, you will build, wire, and program a miniature four-joint robotic arm — a hands-on introduction to real robotic engineering using the same concepts found in industrial systems.

1.3 Where Robotic Arms Are Used

Robotic arms have been transforming industries for decades, and their applications keep expanding into new and surprising territory.

1.3.1 Classical Applications

For many years, robotic arms were the unsung heroes of industrial manufacturing. In auto-

mobile factories, they weld metal chassis with sparks flying at inhuman speed, paint car bodies with flawless consistency, and assemble components along production lines that never stop. In warehouses, they sort and package thousands of items per hour. On space missions, robotic arms mounted on shuttles and space stations have been used to move satellites, repair equipment, and assist astronauts working in the vacuum of space.

Figure 5: Robotic automation in automobile painting and warehouse packaging.

1.3.2 Modern Applications

Today, robotic arms are entering fields that would have seemed unimaginable just a generation ago. In hospitals, surgical robotic systems guide instruments through incisions smaller than a centimetre, giving surgeons a level of precision and control that reduces patient recovery time from weeks to days. In semiconductor factories, robotic arms assemble microchips so microscopic that a single human fingerprint would destroy them. In research laboratories, they handle dangerous materials and perform repetitive experimental procedures with tireless accuracy. And in ongoing space exploration programs, robotic arms on planetary rovers conduct geological experiments and collect samples millions of kilometres from any human operator.

Figure 6: Robotic assistance in medical surgeries.

1.4 World Robot Olympiad: Where Student Robotics Goes Global

WRO is an annual international robotics competition held across over 85 countries. Teams of two or three students build and program autonomous robots to solve engineering challenges — no remote controls, no human guidance once the match begins. The robot must sense, decide, and act entirely on its own. Founded in 2004, WRO has grown into one of the most prestigious youth robotics events in the world, with students competing through local, national, and international stages every year.

Figure 7: WRO Banner

1.4.1 Categories

RoboMission is the flagship track. Teams build a LEGO-based autonomous robot and complete as many scored missions as possible on a themed game field within two and a half minutes. Missions require navigation, object handling, sorting, and precise placement — genuine engineering under pressure. Age divisions are Elementary (8–12), Junior (11–15), and Senior (14–19).

Future Engineers challenges teams to build a self-driving model car that autonomously navigates a track, avoids obstacles, and com-

pletes parking tasks — open hardware, any microcontroller, any sensors. Direct preparation for real autonomous vehicle engineering.

Future Innovators is a project-based track where teams identify a real community problem and build a robot-based solution, presenting their work to a panel of judges.

RoboSports features teams competing head-to-head in a robot game that changes format each year.

1.4.2 WRO 2026: Robots Meet Culture

The theme for WRO 2026 is **Robots Meet Culture**. Teams are challenged to look beyond purely technical applications and explore how robotics can be used to express, preserve, and share cultural heritage, arts, and human stories. This is an invitation to think creatively — your robot is not just a machine, it is a storyteller.

The 2026 India season runs across four progressive levels:

- **Level 1 — Virtual Championship:** Future Innovators teams submit project presentations online for national ranking. Top 400 teams advance to regional events.
- **Level 2 — Regional Championship:** In-person events held across major Indian cities through July and August 2026.
- **Level 3 — National Championship:** The best teams from regional events compete nationally for the right to represent India.
- **Level 4 — International Finals:** National champions travel to the host country to compete against teams from over 85 nations.

Full 2026 rules for all categories — including game field specifications, scoring systems, hardware restrictions, and judging rubrics — are published at *wroindia.org/category-rules-2026*. Registration is open at *registration.wroindia.org*. Note that for 2026, each school is eligible for one free team registration under the Future Innovators category, and the first 15 teams registering under Future Engineers receive free entry.

1.4.3 NCSM and NCSD: Supporting Your WRO Journey

The **National Council of Science Museums (NCSM)** is a premier institution under the Ministry of Culture, Government of India, operating a nationwide network of science museums, science centres, and specialised units dedicated to science communication and education. The **National Science Centre, Delhi (NCSD)** is one such unit under NCSM, among many others spread across the country, each contributing to NCSM's broader mission of bringing science and technology closer to students and communities.

Through NCSM's **Innovation Hubs**, students gain access to WRO-supported robotics kits, hands-on training in robot design and programming, and structured guidance through the conceptualisation and development of competition strategies. NCSM/NSCD supports student teams through registration fee sponsorship and provides the infrastructure, equipment, and mentorship needed to take a workshop-level idea all the way to a national competition entry — removing financial and logistical barriers that might otherwise prevent talented students from competing.

The skills you are building here are the same foundations WRO competitors build on, and NCSM's Innovation Hubs serve as the bridge between learning robotics and competing in it at a national and international level.

For information on NCSM's WRO support programme, training schedules at your nearest Innovation Hub, and details on sponsored participation, visit *ncsm.gov.in/activities-main/events/world-robot-olympiad-2026* or speak to your workshop instructor.

Workshop: Interested in competing in WRO 2026? Talk to your instructor about NCSM's sponsored participation pro-

gramme. The official India competition portal is *wroindia.org* and the international body is at *wro-association.org*.

1.5 What This Module Will Teach You

This learning module takes you on a structured journey through the world of robotics. It is designed to build your understanding progressively — from the big picture down to the fine details.

You will begin by exploring the **theory** that underpins every robotic system: how robots are structured mechanically, how electrical systems power and control their movements, and how programming transforms hardware into an intelligent machine. This theoretical foundation is important because it gives you the understanding to not just build things, but to truly know **why** they work — and how to fix them when they do not.

Once the theory is solid, the module transitions into a complete, step-by-step **practical guide** where you will assemble the mechanical structure of a robotic arm, wire up its electronics, and write the programs that bring it to life.

By the end, you will be able to look at a robotic system — whether in a factory, a hospital, or a space mission — and understand the engineering principles making it work. More importantly, you will have built one yourself.

Let us begin!

2 Fundamentals of Robotics

A robot is far more than a mechanical toy or a remote-controlled gadget. A true robotic system is a carefully engineered combination of mechanical structures, electronic circuits, sensors, actuators, and programmable intelligence — all working together as one integrated machine.

Modern robotics draws from an impressive range of disciplines: mechanical engineering gives robots their physical form and movement; electronics and electrical systems provide power and signals; computer programming supplies instructions and logic; control systems ensure accurate, stable behaviour; and increasingly, artificial intelligence allows robots to learn and adapt to their environment.

Depending on their design, robots can operate fully autonomously — making every decision on their own — or under human supervision, with a person guiding their actions from a distance. Industrial robotic arms, autonomous vehicles, surgical robots, planetary rovers, and underwater drones are all built using the same fundamental principles you are about to learn.

2.1 How a Robot Thinks and Acts

Before diving into components and classifications, let us answer the most important question first: **how does a robot actually work?**

Every robotic system, no matter how simple or complex, follows a single fundamental cycle. It senses the world around it, processes that information to make a decision, and then acts on that decision through physical movement or output. This three-step loop — **sense, process, act** — is the heartbeat of every robot ever built.

Think of it like this: imagine you are riding a bicycle and you notice a pothole in the road ahead. Your eyes (sensors) detect the hazard. Your brain (processing unit) decides to steer around it. Your arms and legs (actuators) carry out that decision. A robot does exactly the same thing, just with cameras and distance sensors instead of eyes, a microcontroller instead of a brain, and motors instead of muscles.

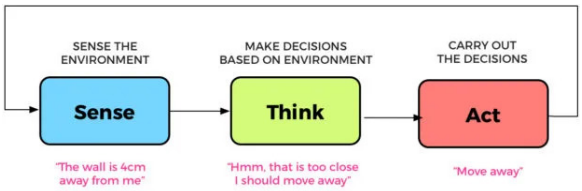


Figure 8: Functional flow of a robotic system.

A camera or distance sensor feeds information into the system. A microcontroller such as Arduino receives that information, runs through its programmed instructions, and decides what to do. Then motors or servo motors carry out the physical response. This cycle happens continuously — often hundreds of times per second — allowing a robot to respond to a changing environment in real time.

All modern robots, from industrial robotic arms to self-driving cars, are built around this same fundamental architecture.

2.2 The Building Blocks of a Robot [1]

Now that you understand how a robot works conceptually, let us look at the individual components that make it happen. Think of a robot as a miniature version of the human body — each mechanical and electronic part has an analogue in your own anatomy.

2.2.1 Links

The rigid structural parts of a robot are called **links**. These form the skeleton of the robot, providing the physical framework that everything else is attached to and moves around.

In a robotic arm, each solid segment between two joints is a link. Links must be strong enough to handle mechanical forces during movement, but also as lightweight as possible — a heavier arm requires more powerful motors, which in turn require more power and more robust electronics. Engineers carefully balance strength and weight in every link they design.

2.2.2 Joints

If links are the bones, then **joints** are where the action happens. A joint is the connection between two links that allows them to move relative to each other.

There are two fundamental types of joints in robotics. A **Revolute Joint (R)** allows rotational motion — one link pivots around the other, like the hinge of a door or the rotation of your elbow. A **Prismatic Joint (P)** allows linear sliding motion — one link slides along the other in a straight line, like the piston in an engine.

The combination and arrangement of revolute and prismatic joints determines the entire movement repertoire of a robot. Most robotic arms use revolute joints almost exclusively, since rotational motion is highly versatile and closely mimics the movement of a human arm.

2.2.3 Manipulator

The complete mechanical structure formed by links and joints together is called the **manipulator**. In robotic arms, the manipulator is the long, jointed structure that positions the working end of the arm in space. Industrial robotic arms are essentially very sophisticated manipulators, engineered to position their endpoints with extraordinary precision and repeatability.

2.2.4 Wrist

Near the end of the manipulator, just before the working tool, is the **wrist**. The wrist provides fine control over orientation — allowing the robot to tilt, rotate, or angle its end tool to approach objects from different directions.

This is especially important in tasks like surgery, welding, or assembly, where the approach angle matters as much as the position.

2.2.5 End-Effector

The **end-effector** is the tool attached at the very tip of the robotic arm. It is the part that directly interacts with the world and performs the actual work. Just as a human hand can hold a pen, grip a ball, or press a button, a robotic end-effector is designed and swapped out depending on the task at hand. Grippers pick up objects; welding torches join metal; vacuum suction tools lift flat surfaces; surgical instruments perform precise medical procedures.

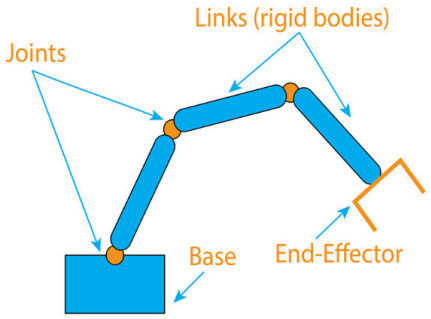


Figure 9: Components of a robotic arm with labeled parts.

See Figure 9 for a labeled view of a typical robotic arm with its components identified.

2.2.6 Actuators

Actuators are the muscles of the robot — the components that convert electrical energy into mechanical movement. Without actuators, a robot would be nothing but a rigid sculpture.

Common actuators used in robotics include servo motors (which rotate to precise angles), DC motors (which spin continuously and are often used in wheeled robots), stepper motors (which move in precise incremental steps), pneumatic cylinders (which use compressed air to push or pull), and hydraulic systems (which use pressurised fluid to produce very large forces).

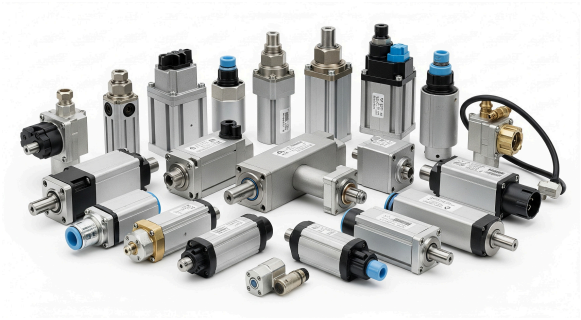


Figure 10: Various types of actuators used in robotics.

2.2.7 Sensors

If actuators are the muscles, then **sensors** are the senses. Sensors allow the robot to gather information about both its own state and the world around it.

Sensors can measure an enormous range of physical quantities: position and orientation, distance to obstacles, speed and acceleration, applied force and pressure, temperature, light intensity, sound, and much more. All of this information flows back to the controller, where it is used to make decisions and adjust the robot's behaviour.

Figure 11: Various types of sensors used in robotics.

2.2.8 Controller

Finally, the **controller** is the brain of the robot. It receives data from sensors, runs the programmed instructions, makes decisions, and sends commands to the actuators to perform the required motion or action.

Modern robots commonly use microcontrollers — small, programmable computer chips — as their controllers. Microcontrollers such as Arduino are popular in educational

and hobbyist robotics, while professional industrial robots use more powerful industrial control systems.

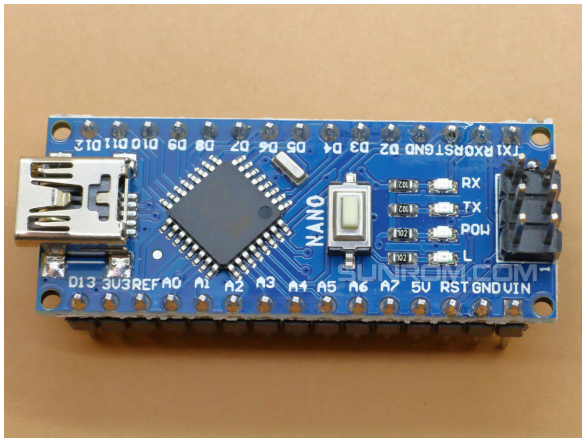


Figure 12: Arduino Nano microcontroller — a common robot brain.

Together, these components — links, joints, manipulator, wrist, end-effector, actuators, sensors, and controller — form the complete robotic system. Remove any one of them and the robot cannot function properly. This is what makes robotics such a rich and interdisciplinary field: it demands expertise from mechanics, electronics, and computer science all at once.

2.3 A Short History of Robotics [1]

The dream of building machines that move and work like living creatures is ancient — appearing in myths and stories from civilisations thousands of years old. But the engineering reality of modern robotics is much more recent, shaped by a series of remarkable innovations across the twentieth century.

The story begins around 1938, when one of the first machines capable of controlling the position of a tool — used for spray painting — was developed. Then, during World War II, engineers built what they called **teleoperators**: remotely controlled mechanical arms that allowed workers to handle radioactive and hazardous materials from a safe distance. This idea — controlling a machine from far away to do dangerous work — remains central to robotics to this day.

Around the same time, another critical technology was maturing: **Computer Numerical Control (CNC)**. CNC systems allowed machines to be guided by programmed numerical instructions rather than by a human operator's hands, dramatically improving the precision and repeatability of industrial manufacturing.

These streams of invention converged in 1954, when George Devol designed one of the first truly programmable robotic systems. A few years later, Joseph Engelberger — inspired by Devol's work — founded Unimation, the world's first commercial robotics company, and began manufacturing robotic arms called **Unimates** for use in factories. For this pioneering work, Engelberger is widely regarded as the "Father of Robotics."

Throughout the 1960s and 1970s, robotic arms became increasingly established in the automobile industry, performing repetitive tasks like welding and painting with a consistency no human worker could sustain. The 1980s saw a dramatic expansion of the industry, fuelled by major investments in factory automation across manufacturing economies worldwide.

Today, robotics is evolving faster than ever before. Artificial intelligence, advanced computer vision, and miniaturised electronics are making robots smarter, more adaptable, and capable of working safely alongside human beings in ways that were science fiction just a decade ago.

2.4 Types of Robots [1]

Not all robots are created equal. Engineers classify robots in several different ways depending on their design, control system, movement capability, and intended application. Understanding these classifications helps

you quickly identify what a robot can do and how it is built.

2.4.1 Classification by Capability

One common way to group robots is by how they are controlled and how intelligently they operate.

Manual handling devices are directly operated by a human in real time — more of a tool than a true robot. **Fixed sequence robots** repeat the same set of actions over and over, exactly as programmed, with no ability to vary or adapt. **Programmable robots** can have their sequence of actions reprogrammed — you can change what they do by updating their software. **Playback robots** record motions guided by a human operator and then reproduce them automatically. **Numerically controlled robots** follow precise motion instructions expressed as numbers and coordinates. And **intelligent robots** represent the frontier of the field — systems capable of sensing their environment, making complex decisions, and adapting their behaviour to changing conditions.

2.4.2 Classification by Geometry

Robots can also be classified by the physical structure of their manipulator — essentially, the shape of their skeleton.

Serial manipulators have links connected in a single open chain, one after another, from the base to the end-effector. This is the most common architecture for robotic arms, and it closely resembles the structure of a human arm. **Parallel manipulators** use a closed-loop structure where multiple chains connect the base to the moving platform simultaneously — this makes them exceptionally rigid and precise, though at the cost of a smaller workspace. **Hybrid manipulators** combine elements of both.

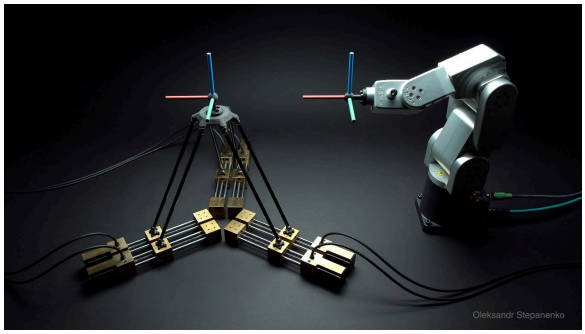


Figure 13: Serial (right) and parallel (left) manipulators.

2.4.3 Classification by Workspace

The **workspace** of a robot is the complete set of positions that its end-effector can reach. Engineers distinguish between the **reachable workspace** — every position the end-effector can get to in at least one configuration — and the **dexterous workspace** — positions where the end-effector can arrive with full freedom of orientation. The workspace of a robot depends on the number, type, and arrangement of its joints, as well as the lengths of its links.

2.4.4 Classification by Actuation

Robots are also grouped by the type of actuators that power their movement.

Electric robots use electric motors as their actuators. They are the most common type in use today — clean, precise, relatively quiet, and easy to control with software. **Hydraulic robots** use pressurised fluid and are capable of producing enormous forces, making them suitable for heavy industrial applications. **Pneumatic robots** use compressed air and are favoured in applications requiring fast, repetitive motions.

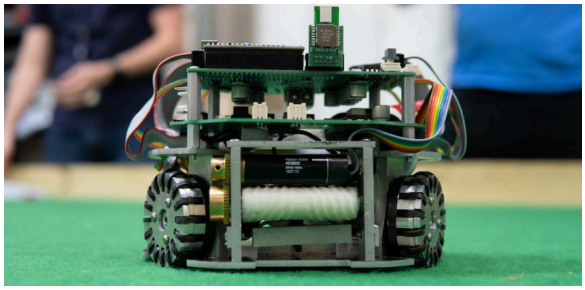


Figure 14: Electric robot with electrical drive components.

Figure 15: Hydraulic robot with hydraulic drive components.

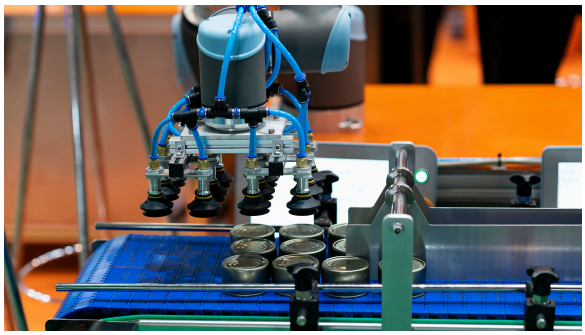


Figure 16: Pneumatic robot with pneumatic drive components.

2.4.5 Classification by Control System

Another important classification concerns how a robot controls its own movement.

Servo robots, also called **closed-loop control** robots, use sensors to continuously monitor their position and compare it to the desired position, automatically correcting any errors. This is like steering a car — you constantly adjust based on what you see. **Non-servo robots**, or **open-loop control** robots, follow their programmed instructions without any feedback — they simply execute commands and assume everything went according to plan.

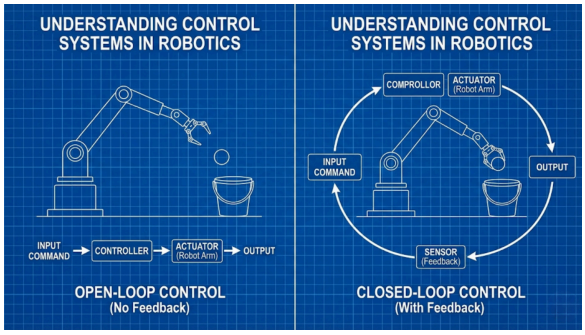


Figure 17: Open-loop (left) and closed-loop (right) control systems.

Within servo robots, a further distinction is made between **point-to-point robots**, which move between a set of predefined positions without concern for the exact path taken between them, and **continuous path robots**, which carefully control every point along their trajectory — essential for tasks like painting or welding where the path itself matters.

2.4.6 Classification by Application

Finally, robots are often described simply by what they do. Industrial robots manufacture products; medical robots assist surgeons; agricultural robots tend crops; inspection robots examine pipelines, bridges, and aircraft; space robots explore planets and service satellites; and search-and-rescue robots navigate disaster zones to find survivors. Many robotic arms used in industry are designed to mimic the range of motion of a human arm, and for this reason they are sometimes called **anthropomorphic robots**.

> ***Workshop:*** The robotic arm in this workshop is a serial, electric, open-loop manipulator with revolute joints — a category that covers a huge proportion of all robotic arms in real-world use.

2.5 Degrees of Freedom: Understanding Robot Motion

One of the most important concepts in all of robotics is **Degrees of Freedom**, often abbreviated as **DoF**.

Degrees of freedom describes the number of independent movements a robotic system can perform. Every joint that allows a separate movement adds one degree of freedom to the robot. Think of it this way: if a joint can only rotate, that is one DoF. If another joint can also rotate (but about a different axis), that adds a second DoF, and so on.

Your own body is a powerful example. The shoulder joint allows rotation in multiple directions — that alone contributes several degrees of freedom. The elbow adds one more (bending). The wrist adds more still (rotating, bending, tilting). Each independent motion your arm can make corresponds to one degree of freedom. Taken all together, the human arm has an impressive number of DoF — which is why our hands are capable of such a staggering variety of movements.

In a robotic system, more degrees of freedom means greater flexibility and a larger workspace — the robot can reach more positions, from more angles. Fewer degrees of freedom means a simpler, cheaper, and easier-to-control system, but with more limited capability. A simple robotic arm might have three DoF; a full industrial six-axis robot arm can reach almost any position and orientation within its workspace. Advanced humanoid robots have DoF in the dozens.

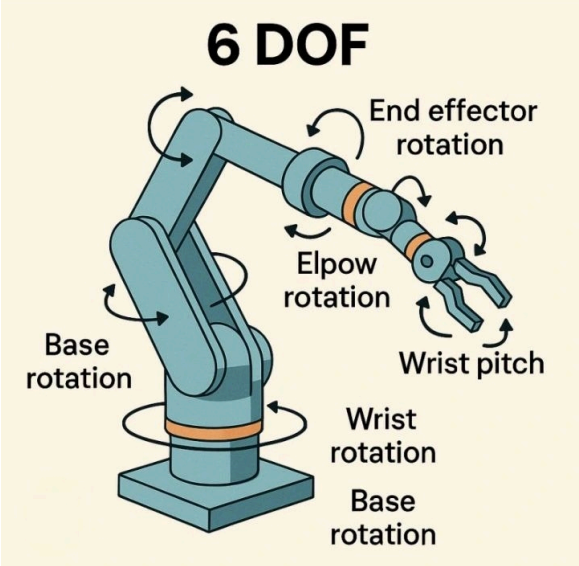


Figure 18: Degrees of freedom in a robotic arm.

The concept of degrees of freedom guides almost every design decision made when building a robot — how many joints to include, what types to use, and where to place them. It is the language that engineers use to describe and compare robotic systems across the entire field.

Workshop: The robotic arm you will build in this workshop has **3 degrees of freedom** — three revolute joints that position the arm in space — plus a fourth motor dedicated solely to opening and closing the end-effector claw. The claw motor does not add a positional DoF; it is an actuated gripper. This gives the arm the same fundamental architecture as many entry-level industrial and educational robotic arms worldwide.

3 Mechanical Systems in Robotics

Every robot that interacts physically with the world needs a body — a mechanical structure that provides shape, support, and the ability to move. No matter how sophisticated the electronics or clever the programming, a robot cannot do useful work without a well-designed mechanical foundation.

In robotics, mechanical design covers everything from the overall structure of links and joints to the small screws that hold components in place. Getting this right matters enormously. A poorly designed mechanical system will flex, vibrate, or break under load — no amount of clever software can compensate for a structure that physically cannot do what is asked of it. Good mechanical design brings together strength, precision, lightness, and smooth motion — and balancing all four at once is what makes mechanical engineering both challenging and deeply satisfying.

In this chapter, you will explore the major mechanical elements found in robotic systems: how links and joints create movement, how gears transfer and transform forces, and how fasteners hold everything together reliably.

3.1 Links: The Skeleton of a Robot

A **link** is any rigid structural member in a robotic system — a solid part that does not flex or bend during normal operation. Links form the skeleton of the robot, providing the physical framework that all other components are mounted to and that gives the robot its shape.

In a robotic arm, each solid segment between two adjacent joints is called a link. The base of the arm is sometimes considered a fixed link (it does not move), while all subsequent links rotate or translate relative to each other.

The design of a link involves careful trade-offs. It must be strong enough to carry the loads placed on it — the weight of subsequent links, the end-effector, and any object being handled — without bending or breaking. At the same time, it should be as light as possible, because every gram of extra weight the arm must carry reduces its speed, increases the load on its motors, and demands more power from its electrical supply. Engineers select materials such as aluminium alloys, carbon fibre composites, or high-strength plastics depending on the application. In educational and hobbyist robotics, 3D-printed plastic links are extremely common because they are cheap, customisable, and strong enough for light-duty use.

3.2 Joints: Where Motion Happens

If links are the bones, joints are where everything comes alive. A **joint** connects two links together and defines how they can move relative to each other.

The two most fundamental types of joints in robotics are the **revolute joint** and the **prismatic joint**.

A **Revolute Joint (R)** permits rotational motion: one link pivots around an axis fixed to the other link. This is the most common type of joint in robotic arms — it is mechanically simple, produces large angular displacements with relatively small actuator movement, and closely resembles the joints of the human arm. Your elbow is a good example: it rotates your forearm relative to your upper arm in a single arc of motion.

A **Prismatic Joint (P)** permits linear sliding motion: one link slides along a fixed axis relative to the other. Rather than rotating, the joint extends or retracts, changing the distance between the two connected links. Linear actuators, telescoping segments, and sliding rails all implement prismatic motion. Prismatic joints appear frequently in gantry robots and CNC machines.

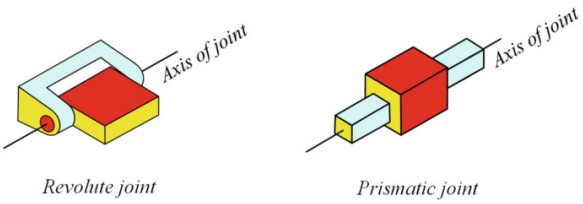


Figure 19: Revolute (R) and Prismatic (P) joints. [1]

In practice, almost every robotic arm you will encounter — from the simple educational kind to the massive industrial systems in car factories — is built almost entirely from revolute joints. Their simplicity, compactness, and versatility make them the default choice.

3.3 The Robotic Arm Structure

A robotic arm brings links and joints together in a chain, starting from a fixed base and ending at the end-effector. Each link is connected to the next by a joint, and each joint allows a certain degree of motion. The combined result is a system that can position its tip in three-dimensional space across a wide range of positions and orientations.

Understanding the structure of a robotic arm is easiest when you think of it as an engineering version of your own arm and hand. The first link from the base corresponds roughly to the upper arm; the second to the forearm; and then there may be additional links corresponding to the wrist and hand. Each joint between these links corresponds to a joint in your body.

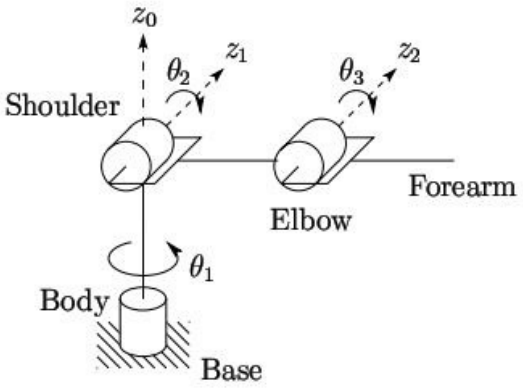


Figure 20: Schematic diagram showing links, joints, and end-effector in a robotic arm.

The end-effector — the tool at the tip of the arm — is the part that actually interacts with objects in the world. It is replaceable: swap out a gripper for a welding torch, or a welding torch for a surgical instrument, and you have a completely different robot in terms of capability, even though the underlying mechanical arm is unchanged. This modularity is one of the great engineering strengths of the robotic arm design.

3.4 Gears: Multiplying Strength and Precision

One of the most elegant mechanical inventions ever made is the **gear** — and you will find them inside almost every actuator in a robotic arm.

A gear is a toothed wheel that meshes with another toothed wheel (or a toothed rack) to transfer rotational motion from one shaft to another. When two gears of different sizes are meshed together, something very useful happens: the smaller gear (the driver) turns faster but with less torque, while the larger gear (the driven) turns slower but with more torque. This relationship is called the **gear ratio**, and it is one of the most powerful tools mechanical engineers have.

In robotics, gears serve several important purposes. They allow a small, high-speed electric motor to produce large amounts of torque by

stepping down its speed through a gear train. They improve the precision of movement — because each motor rotation produces a much smaller rotation at the output, any small error in the motor position is correspondingly reduced. And they allow engineers to transmit motion around corners, between parallel shafts, or through tight mechanical arrangements that would be impossible to achieve otherwise.

Figure 21: A variety of gear types used in mechanical systems.

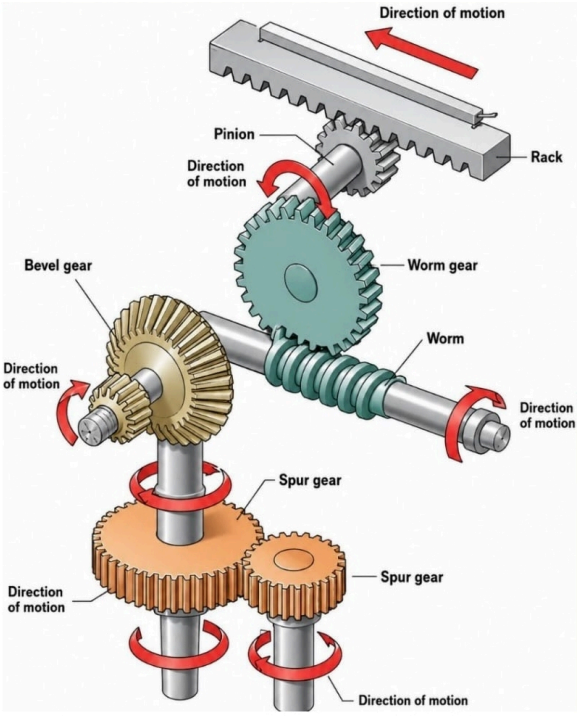


Figure 22: Transfer of energy and motion through meshing gears.

In servo motors — the actuators used in most educational and hobby robotic arms — a small electric motor spins at very high speed. Inside the servo housing, a compact gear train (often called a **gearbox** or **gear reduction**) reduces this speed dramatically while multiplying the torque. The result is an output shaft that moves slowly and precisely, with enough rotational force to move a robot joint against gravity and mechanical resistance. Without gears, the tiny motor inside a servo would spin uselessly fast without the strength to move anything.

3.5 Fasteners: Holding It All Together

Every piece of a robot's mechanical structure must be attached to every other piece reliably. This is the job of **fasteners** — hardware components such as screws, bolts, and nuts that join parts together mechanically.

In robotics, the most commonly used fasteners are small **metric screws**, designated by the letter **M** followed by a number indicating the diameter in millimetres. The full specification of a metric screw is written in the format:

M3×12×0.5

This means:

- **M3** — the screw has a 3 mm shaft diameter
- **12** — the screw is 12 mm long
- **0.5** — the thread pitch (distance between adjacent threads) is 0.5 mm

In many practical situations, the pitch value is omitted because standard coarse-pitch screws are assumed.

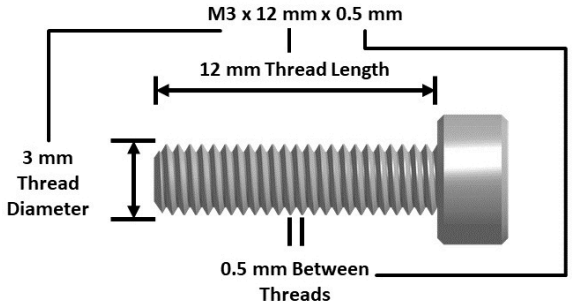


Figure 23: Reading the specification of a metric screw.

Screws also differ in the shape of their head — which determines what tool is used to drive them — and in how they sit in their hole. Common **drive types** include the **Phillips head** (the familiar cross-shaped recess), the **slotted head** (a single straight slot), the **hex socket** (a hexagonal recess driven by an Allen key, providing excellent grip and torque), and the **Torx head** (a six-pointed star shape, common in precision and high-torque applications).

Mounting styles vary too. **Countersunk screws** are designed to sit flush with the surface of the material — the head tapers into a cone that fits into a matching recess, leaving a flat surface. **Counterbore screws** sit below the surface inside a cylindrical pocket, where a cap covers them. **Pan head screws** have a rounded, protruding head that sits on the surface, making them easy to install and remove.

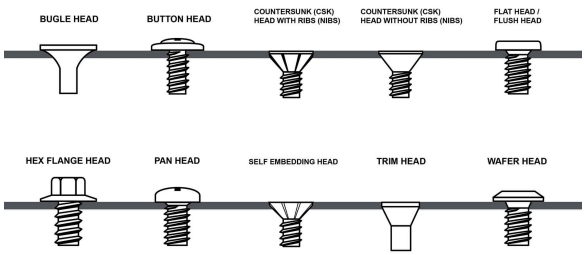


Figure 24: Various screw head shapes and drive types.

Figure 25: Various screw types for different mounting applications.

Choosing the right fastener for each location in a robotic assembly requires thinking about the available space, the load the joint must carry, the material being fastened, and how often the joint may need to be disassembled for maintenance. In 3D-printed assemblies — which are common in educational robotics — fasteners must be chosen and installed with particular care, since plastic threads are softer and easier to strip than metal ones.

> ***Workshop:*** The robotic arm in this workshop uses M2 metric screws throughout its assembly. You will find specific screw sizes called out at each assembly step in the practical guide.

4 Electrical and Control Systems

A robot with a perfect mechanical structure but no electronics would be nothing more than a very elaborate sculpture. It is the electrical and control systems that give a robot its ability to sense, decide, and act — to be truly alive as a machine.

The electrical and control layer of a robot is responsible for three things: gathering information about the world through sensors, processing that information and making decisions through a controller, and carrying out those decisions through actuators. These three functions map directly onto the sense–process–act cycle you learned about in the previous chapter. In this chapter, you will explore each of those functions in depth — the hardware that implements them, how they work, and why they are designed the way they are.

4.1 Sensors: The Robot's Senses

Your eyes let you see, your ears let you hear, your skin lets you feel temperature and pressure, and your inner ear gives you your sense of balance. A robot needs equivalent capabilities — a set of devices that let it perceive and measure its environment. These devices are called **sensors**.

A sensor converts a physical quantity in the real world — distance, light intensity, temperature, force — into an electrical signal that the robot's controller can read and interpret. Without sensors, a robot is blind to the world around it. It can only follow pre-programmed instructions blindly, with no ability to respond to unexpected changes in its environment.

Figure 26: Various types of sensors used in robotics.

The range of sensors used in modern robotics is remarkable. **Distance sensors** (such as ultrasonic and infrared sensors) measure how far away an object is. **Camera systems** capture images or video that computer vision algorithms can analyse to identify objects, read text, or navigate environments. **Force and torque sensors** tell a robot how hard it is pushing or pulling. **Encoders** measure the precise rotational position and speed of a motor shaft. **Temperature sensors** monitor heat levels to prevent overheating. **Inertial measurement units (IMUs)** measure acceleration and angular velocity to track a robot's orientation and motion in space.

The information from all these sensors flows back to the robot's controller, where it is combined with the programmed instructions to determine what action to take next. A robot that uses sensor feedback to continuously adjust its behaviour is far more capable and reliable than one that simply executes fixed commands.

4.2 Actuators: The Robot's Muscles

While sensors provide input, **actuators** provide output — they are the components that convert electrical energy into physical movement. Without actuators, a robot can sense and think all it likes, but it cannot do anything. Actuators are the muscles that make a robot act on the world.

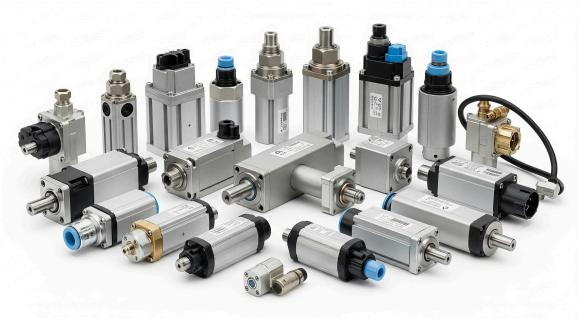


Figure 27: Various types of actuators used in robotics.

The most common actuators in robotics are electric motors, which come in several important varieties. **DC motors** spin continuously when powered, and their speed and direction can be controlled electrically. They are widely used in wheeled robots and conveyor systems. **Stepper motors** divide a full rotation into a large number of precise equal steps, making them ideal for applications requiring accurate position control without a separate position sensor — they are common in 3D printers and CNC machines. **Servo motors** add a position sensor and a feedback control circuit to a DC motor, allowing them to rotate to and hold a precise angle — they are the actuator of choice for robotic arms.

Beyond electric motors, **pneumatic actuators** use compressed air to drive linear pistons and are valued for their speed and force in industrial automation. **Hydraulic actuators** use pressurised fluid and can produce enormous forces, making them the actuator of choice in heavy industrial robots, construction equipment, and high-power applications.

For most educational and hobby robotic arm projects, servo motors are the go-to actuator because they combine precision, simplicity, and affordability in a compact package.

4.3 Servo Motors: Precision in Every Degree

Servo motors deserve a deeper look, because they are arguably the most important actuator in the world of educational and hobby robotics.

What makes a servo motor special compared to an ordinary DC motor? A regular motor just spins — fast, continuously, and without any built-in sense of where it is. A servo motor, by contrast, can rotate to a **specific angle** and then hold that position with force, even if something tries to push it away. This precise, position-controlled behaviour is exactly what a robotic arm joint needs.

Think about what you need a robotic arm to do: move joint 2 to exactly 90 degrees, hold it there while the end-effector grips an object, then move to 45 degrees. A DC motor alone cannot do this reliably. A servo motor can — it is engineered specifically for this kind of precise, commanded motion.

4.3.1 Inside a Servo Motor

Despite their small size, servo motors contain several cleverly integrated components working together.

Inside the housing you will find a small DC motor — the actual source of rotational power. A **gear reduction train** (a compact system of interlocking gears) slows down the motor's high-speed rotation and multiplies its torque, producing an output shaft that turns slowly but with considerable strength. A **position sensor** (typically a small rotary potentiometer) continuously measures the current angle of the output shaft. And an **electronic control circuit** ties everything together, reading the position sensor and controlling the motor to achieve and maintain the commanded angle.

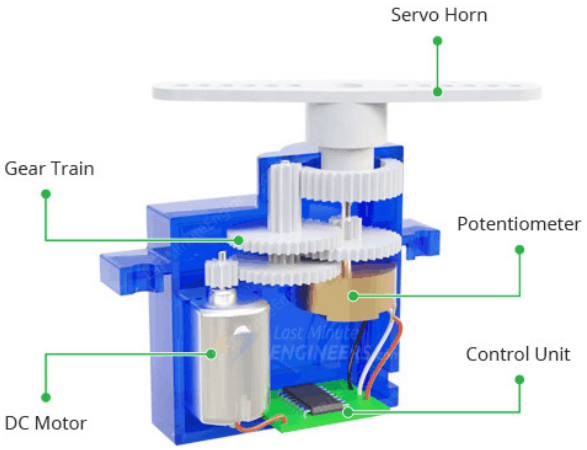


Figure 28: Internal components of a typical servo motor.

4.3.2 How a Servo Motor Works

A servo motor operates using an internal **closed-loop control system**. When it receives a command to move to a certain angle, the internal control circuit compares the commanded angle with the current angle (as reported by the position sensor). If they differ, the circuit drives the motor in the direction that will reduce the error. As the output shaft approaches the target angle, the motor slows and eventually stops exactly where commanded. If an external force then tries to disturb the shaft from that angle, the servo immediately detects the error and applies corrective force to resist it — this is called **holding torque**.

This entire process happens extremely fast, creating the impression of instantaneous, locked positioning. Even when a robotic arm using servo motors is operating as an open-loop system at the top level (meaning the arm controller does not read feedback from external sensors), each individual servo motor is internally running its own closed-loop position control.

4.3.3 Rotational Limits

Standard hobby servo motors do not rotate freely through 360 degrees like a DC motor. They have a limited rotation range — typically **0° to 180°**, though some models offer a slightly narrower or wider range. This mechanical limit is intentional: it keeps the position sensor's readings unambiguous and allows the internal control circuit to know exactly where the shaft is at all times. Some special servos called **continuous rotation servos** sacrifice angle control to allow unlimited rotation, but these are not position-controlled and behave more like ordinary motors.

4.3.4 Analog and Digital Servo Motors

Servo motors come in two main electrical varieties. **Analog servo motors** use a simpler internal circuit that updates the motor control at a lower rate. They are less expensive and perfectly adequate for many applications. **Digital servo motors** use a faster, microprocessor-based internal controller that updates far more rapidly — delivering quicker response, better holding torque, and more precise positioning. Digital servos are preferred in demanding applications such as competitive robotics and advanced automation.

4.3.5 Servo Motor Pin-Out

A standard servo motor connects through a 3-wire cable with a 3-pin connector. The brown or black wire is GND (Ground), the red wire is V+ (Power Supply, typically 4.8-6 V), and the orange, yellow, or white wire is the PWM signal line that controls the servo position.

Always connect the wires correctly. Incorrect wiring or voltage can permanently damage the servo's internal control circuit. Although the colour convention is widely used, some manufacturers may use different wire colours, so checking the servo datasheet is recommended.

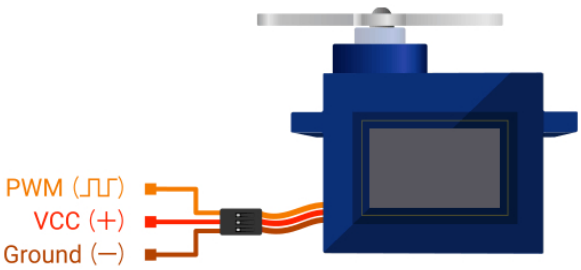


Figure 29: Servo motor connector description.

4.3.6 Controlling a Servo with PWM

Servo motors are controlled using an electrical signal technique called **Pulse Width Modulation**, or **PWM**.

PWM works by rapidly switching a digital signal between a HIGH (on) state and a LOW (off) state. Rather than varying the voltage level to communicate information, PWM varies the **width** — the duration — of each ON pulse. A microcontroller can generate this pattern easily and precisely, making PWM an ideal way to communicate a desired angle to a servo.

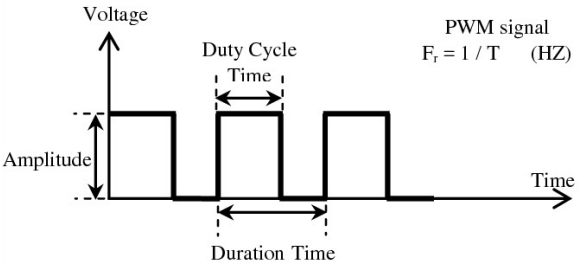


Figure 30: Basic representation of a PWM signal and its parameters.

A PWM signal is characterised by its **frequency** (how many pulses occur per second), its **pulse width** (how long each pulse stays ON), and its **duty cycle** (the percentage of each period that the signal is ON).

For standard hobby servo motors, the control signal runs at approximately **50 Hz** — meaning one pulse arrives every 20 milliseconds. The position the servo moves to is determined by the width of that pulse: a pulse of approximately **1 millisecond** commands the servo to move to near 0°; a pulse of **1.5 milliseconds** moves it to roughly 90° (the centre); and a pulse of **2 milliseconds** moves it to near 180°. The servo reads each incoming pulse, calculates the commanded angle, and drives its internal motor accordingly.

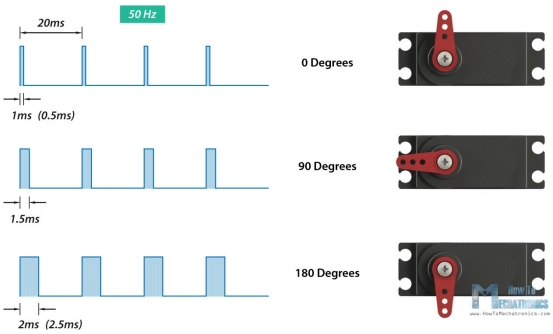


Figure 31: PWM pulse width controlling servo motor angle.

Microcontrollers such as Arduino can generate PWM signals on dedicated output pins, which is exactly how robotic arm controllers command their servo joints.

4.4 Motor Drivers: Managing Power

Here is an important reality about microcontrollers: they are excellent at making decisions and generating control signals, but they are not designed to supply large amounts of electrical current. The output pins of a typical microcontroller can provide only a few milliamps — far too little to directly drive a motor, which may require hundreds of milliamps or more.

This is where **motor drivers** come in. A motor driver sits between the microcontroller and the motor, acting as a powered intermediary. The microcontroller sends a small, low-power control signal to the driver; the driver uses that signal to switch a separate, high-current power supply to the motor in the right way. The microcontroller gives the instruction; the driver supplies the muscle.

Without a proper motor driver, you risk destroying your microcontroller by overloading its output pins. Worse, motors connected directly to a microcontroller may draw so much current during startup or stall that they reset or damage the controller entirely. Motor drivers protect against all of this.

4.4.1 Types of Motor Drivers

Different robots use different motor drivers depending on the type of motors being controlled. **DC motor drivers** (often based on an H-bridge circuit) allow a microcontroller to control the speed and direction of one or more DC motors. **Stepper motor drivers** generate the precise, timed sequences of pulses that stepper motors require. **Servo motor drivers** generate stable PWM signals for multiple servo motors simultaneously. **High-power motor controllers** handle the large currents needed in industrial robots, electric vehicles, and heavy machinery.

4.4.2 Why Use a Dedicated Servo Driver?

Servo motors contain their own internal control circuits, so you might wonder: does a robotic arm really need a separate servo driver module? Can the microcontroller not just generate PWM signals directly?

The answer is: it depends on how many servos you need. A microcontroller can directly generate PWM on a limited number of output pins — typically six to ten on a common Arduino board. For a simple one or two-servo project, direct control works fine. But a robotic arm with four, six, or more joints needs PWM on just as many channels. As the number of servos grows, the microcontroller starts to struggle: it runs out of PWM pins, the processing overhead of generating all those signals occupies valuable computation time, and the timing becomes less accurate because the controller is juggling too many tasks at once.

A dedicated servo driver module solves all of these problems at once. It takes over the task of generating PWM signals entirely, operating independently and freeing the microcontroller to focus on higher-level logic. The microcontroller simply tells the driver "set channel 3 to this angle" via a simple communication interface, and the driver takes care of the rest.

4.4.3 The PCA9685 Servo Driver Module

One of the most widely used and well-supported servo driver modules in robotics and electronics is the **PCA9685 PWM Driver Module**.

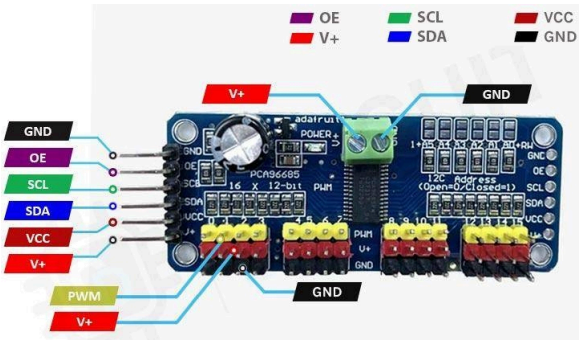


Figure 32: PCA9685 16-channel servo motor driver module.

The PCA9685 is a dedicated PWM controller chip mounted on a convenient breakout board. It can independently generate PWM signals on up to **16 channels simultaneously**, each fully configurable in frequency and pulse width. It communicates with a microcontroller using **I2C** — a widely used two-wire serial communication protocol that requires only two signal wires (a clock line and a data line) regardless of how many modules are on the bus.

Instead of generating 16 separate, carefully timed PWM signals itself, the microcontroller simply sends I2C messages to the PCA9685 specifying which channel to update and what PWM value to set. The PCA9685 handles the signal generation independently, producing stable and accurate pulses at all 16 outputs simultaneously. This dramatically reduces the processing burden on the microcontroller and ensures that all servo signals are generated with consistent, hardware-accurate timing.

4.4.4 PCA9685 Pin Description

The PCA9685 module exposes several groups of pins. The **power pins** include VCC (logic supply, typically 3.3V or 5V from the microcontroller), GND (ground), and V+ (a separate power rail for the servo motors, supplied

by an external power source capable of delivering the current the motors require). The **I2C communication pins** are SDA (serial data) and SCL (serial clock), which connect directly to the corresponding I2C pins on the microcontroller. The **PWM output channels** (numbered 0 through 15) are 3-pin headers — ground, power, and signal — that connect directly to servo motor cables. Each header pin corresponds to the standard servo wire colour convention: **GND** connects to the **brown or black** wire, **V+** connects to the **red** wire, and the **PWM signal** pin connects to the **orange, yellow, or white** wire.

> ***Workshop:*** The workshop robotic arm uses a PCA9685 to control four servo motors across channels 0 through 3. The wiring and programming of this module are covered in detail in the hands-on guide.

4.5 Microcontrollers: The Brain of the Robot

Think about how your own body works. Your brain receives streams of information from your eyes, ears, and skin, processes it all in fractions of a second, decides what to do, and sends signals to your muscles to carry out those decisions. A robot needs something analogous — a central processing unit that can read sensor data, run the robot's program, make decisions, and send commands to actuators.

This role is filled by the **microcontroller**.

A microcontroller is a small, self-contained programmable computer built for controlling hardware. Unlike a desktop computer, which is designed for general-purpose computing tasks, a microcontroller is optimised for direct interaction with electronic components: reading sensors, toggling outputs, generating timed signals, and communicating with other devices. Microcontrollers are everywhere — in your washing machine, your microwave oven, your car, your alarm clock, your TV remote control. They run the world quietly in the background, and in a robot, they run the show.

In a robotic system, the microcontroller continuously manages all operations. It reads sensor data, evaluates the programmed logic to decide what action to take, and sends the appropriate commands to motor drivers, servo drivers, or other output devices. Without the controller, all the mechanical and electrical components of a robot are just an expensive pile of parts.

4.5.1 Popular Microcontrollers in Robotics

Many different microcontrollers are used in robotics, ranging from the very simple to the extremely powerful. **Arduino** boards (based on Atmel/Microchip AVR chips) are the most widely used in education and hobby robotics, valued for their simplicity and the enormous library of tutorials and community support around them. **ESP32** boards add built-in Wi-Fi and Bluetooth, making them popular for connected robots. **Raspberry Pi Pico** offers a capable microcontroller at very low cost. **STM32** chips are favoured in more advanced and performance-demanding applications. At the upper end, systems like the **Raspberry Pi** or **NVIDIA Jetson** are full embedded computers running Linux, used in robots that require computer vision or AI inference.

4.5.2 Arduino Nano

For compact robotic systems and educational projects, one of the most popular choices is the **Arduino Nano** — a small, breadboard-friendly microcontroller development board based on the **ATmega328P** microcontroller chip.

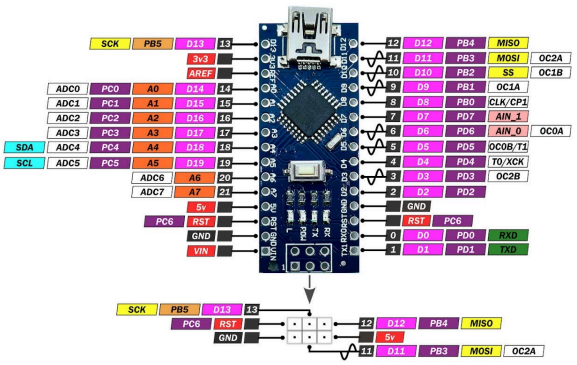


Figure 33: Arduino Nano microcontroller development board.

Despite its compact size (roughly 45 mm × 18 mm), the Arduino Nano packs in everything needed to control a multi-servo robotic system. It has 14 digital input/output pins (6 of which can generate PWM signals), 8 analogue input pins, hardware support for I2C, SPI, and UART serial communication, and runs its program at 16 MHz. It can be powered and programmed through a standard USB cable, and its pin headers plug directly into a standard breadboard — making it ideal for rapid prototyping and educational use.

The key specifications of the Arduino Nano are:

- **Microcontroller:** ATmega328P
- **Operating Voltage:** 5 V
- **Clock Speed:** 16 MHz
- **Digital I/O Pins:** 14 (of which 6 support PWM)
- **Analogue Input Pins:** 8
- **Communication:** UART, I2C, SPI

These features make the Nano well suited to reading sensors, communicating with driver modules via I2C, and generating the control signals needed to run a multi-joint robotic arm.

4.6 Writing Robot Programs

A robot with functioning hardware but no program is useless — it simply sits there waiting for instructions that never come. Programming is the process of providing those instructions: telling the robot exactly what to do, when to do it, and how to respond to different situations.

For Arduino-based systems, programs are written using the **Arduino Framework** — a beginner-friendly programming environment built on top of the C++ programming language. The Arduino Framework provides a simplified way to interact with hardware, wrapping complex low-level operations into easy-to-use functions. It also comes with a huge ecosystem of libraries — pre-written code packages that make it easy to control common devices such as servo drivers, displays, and sensors without having to understand every detail of how they work at the hardware level.

4.6.1 The Arduino IDE

Arduino programs are written, compiled, and uploaded using the **Arduino IDE** (Integrated Development Environment) — a free, cross-platform software application. The IDE provides a text editor for writing code, a compiler that translates your code into machine instructions the microcontroller can execute, and an uploader that sends the compiled program to the board via USB. It also includes a **Serial Monitor** — a small terminal window that lets you send text to and receive text from the running Arduino over USB, which is extremely useful for debugging and for building computer-controlled systems.

An Arduino program is called a **sketch** and is saved with the `.ino` file extension.

4.6.2 Serial Communication

Once a program is running on the Arduino, you often need the microcontroller to **talk to the outside world** — sending data to a computer, receiving commands, or printing debug messages so you can see what is happening inside your code. The primary way Arduino does this is through **serial communication**.

Serial communication is a method of transferring data where bits are sent **one after another** along a single wire — as opposed to

parallel communication, where multiple bits travel simultaneously on multiple wires. Serial is simpler, requires fewer wires, and works reliably over longer distances, which is why it became the dominant method for connecting microcontrollers to computers and to each other.

4.6.2.1 Types and Standards of Serial Communication

Several distinct serial communication standards are used in electronics and robotics. They differ in how many wires they use, how many devices can be connected, and what speeds they support.

UART (Universal Asynchronous Receiver Transmitter) is the most fundamental form of serial communication and the one Arduino uses to talk to your computer over USB. UART uses two signal wires — **Tx** (transmit) and **Rx** (receive) — plus a shared ground. It is called "asynchronous" because both sides agree on a speed in advance (the **baud rate**) rather than sharing a clock signal. The Arduino IDE's Serial Monitor and most serial terminal programs communicate over UART.

I2C (Inter-Integrated Circuit) uses just two wires — **SDA** (data) and **SCL** (clock) — and allows a single controller to communicate with many devices on the same two-wire bus by addressing each one with a unique identifier. It is commonly used for sensors and driver modules such as the PCA9685. I2C is slower than SPI but requires fewer wires and supports many devices on one bus.

SPI (Serial Peripheral Interface) uses four wires — **MOSI**, **MISO**, **SCK**, and **CS** — and is faster than I2C. It is used for high-speed devices such as SD card modules, display screens, and certain sensors. SPI is synchronous: a shared clock wire keeps both sides in step.

RS-232 is an older standard still found in industrial equipment, test instruments, and some legacy systems. It uses higher voltage levels (±12 V) than the 0–5 V TTL logic of microcontrollers, so a level-shifter chip is needed to connect an Arduino to an RS-232 device.

USB (Universal Serial Bus), while not a simple serial line in the traditional sense, is itself based on serial data transfer and is what physically connects your Arduino to your computer. On the Arduino Nano, a small USB-to-UART bridge chip (the CH340 or FT232) converts between the computer's USB connection and the ATmega328P's UART pins, so from the microcontroller's point of view it is simply sending and receiving UART data.

4.6.2.2 Baud Rate

The **baud rate** is the speed of a UART serial connection, expressed in **bits per second (bps)**. Both sides of a UART connection must be set to the same baud rate — if they differ, the received data will be garbled. Common baud rates you will encounter are:

Baud Rate	Typical Use
9600	Default for many beginner sketches and sensor modules.
19200	Moderate-speed sensor and module communication.
57600	Faster data logging and GPS modules.
115200	High-speed Arduino communication; used in this robotic arm project.
250000	Used by 3D printer firmware (Marlin) and high-throughput systems.

Table 1: Common UART baud rates and their typical applications.

Higher baud rates transfer data faster but are more sensitive to cable quality and electrical noise over long distances. For short USB connections between an Arduino and a computer, 115200 baud is reliable and fast enough for real-time control applications.

4.6.2.3 Serial Port Names: Windows and Linux

When you connect an Arduino to a computer, the operating system assigns it a **serial port name** — a software address you use to open a connection to the device.

On **Windows**, serial ports are named **COM** followed by a number — for example `COM3`, `COM7`, or `COM11`. The Arduino IDE automatically detects available COM ports and lists them in the **Tools → Port** menu. You can also find the assigned port in the Windows Device Manager under "Ports (COM & LPT)".

On **Linux** and **macOS**, serial devices appear as files inside the `/dev/` directory. Arduino boards typically appear as `/dev/ttyUSB0` or `/dev/ttyUSB1` (when using a CH340 USB-to-UART chip) or `/dev/ttyACM0` (when using an ATmega16U2 USB interface, as on the Arduino Uno). The Arduino IDE on Linux lists available ports in the same **Tools → Port** menu, reading directly from `/dev/`.

In your Arduino code, you open the serial connection in `setup()` with a single line specifying the baud rate:

```
1  void setup() {
2    Serial.begin(115200);
3  }
```

From that point, you can send data to the computer with `Serial.print()` or `Serial.println()`, and check for incoming data with `Serial.available()` and `Serial.read()` or `Serial.readStringUntil()`. The Arduino IDE's **Serial Monitor** (the magnifying-glass icon in the top-right of the IDE) opens a terminal window connected to whichever COM or `/dev/` port is selected, letting you type commands to the Arduino and read its responses in real time.

4.6.3 The Structure of an Arduino Program

Every Arduino sketch has the same two-part structure. The first part is the `setup()` function, which runs **once** when the microcontroller is powered on or reset. You use `setup()` to initialise hardware, configure pins, start communication interfaces, and do anything else that needs to happen once at the beginning. The second part is the `loop()` function, which runs **continuously and repeatedly** for as long as the board has power. Everything the robot does on an ongoing basis — reading sensors, updating motor positions, checking for incoming commands — goes inside `loop()`.

```
1  void setup() {
2    // Runs once at startup
3  }
4
5  void loop() {
6    // Runs repeatedly, forever
7  }
```

You can think of `setup()` as the moment the robot wakes up and gets ready, and `loop()` as the robot's heartbeat — the continuous pulse of activity that keeps it running.

4.6.4 Variables, Logic, and Decision Making

Real robot programs need to store values and make decisions. **Variables** are named storage locations in memory that hold values — numbers, text, true/false states — that the program can read and modify.

Conditional statements let the program branch and make decisions:

```
1  int angle = 90;
2
3  void setup() {
4  }
5
6  void loop() {
7    if (angle > 45) {
8      // Move one way
9    } else {
10     // Move another way
11   }
12 }
```

This kind of logic is what allows a robot to respond differently to different situations — stopping when an obstacle is detected, adjusting its grip when a force sensor signals resistance, or selecting a different motion sequence based on a command received from a computer.

4.6.5 External Libraries

One of the most powerful features of the Arduino ecosystem is its library system. A **library** is a collection of pre-written code that provides a simple programming interface to a complex hardware device or algorithm. Instead of writing dozens of lines of low-level code to communicate with a servo driver chip, you simply include the relevant library and call its high-level functions.

For the robotic arm system, two libraries are particularly important:

```
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>
```

`Wire.h` is the standard Arduino library for I2C communication — it handles all the low-level detail of sending and receiving data over the two-wire I2C bus. `Adafruit_PWMServoDriver.h` provides a simple interface to the PCA9685 servo driver, allowing you to set any servo channel to any PWM value with a single function call.

4.6.6 Example: Moving a Servo

The following program initialises the PCA9685 servo driver and then repeatedly moves a servo on channel 0 between two positions:

```
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>

Adafruit_PWMServoDriver pwm =
Adafruit_PWMServoDriver();

void setup() {
  pwm.begin();
  pwm.setPWMFreq(50);
}

void loop() {
  pwm.setPWM(0, 0, 102);  // Move to one position
  delay(1000);

  pwm.setPWM(0, 0, 512);  // Move to another position
  delay(1000);
}
```

`pwm.begin()` initialises the driver; `pwm.setPWMFreq(50)` sets the PWM frequency to 50 Hz (the standard for hobby servos); and `pwm.setPWM(channel, on_tick, off_tick)` sets the pulse timing on a given channel. The values 150 and 350 are raw PWM counts that correspond to specific pulse widths — and therefore to specific servo angles.

This tiny program is a prototype of the full robotic arm controller. The same ideas — initialising the driver, setting frequencies, sending PWM values to specific channels — scale up directly to controlling a full four-joint arm.

4.6.7 A Note on the LED Blink Example

One of the very first programs most students write on an Arduino is a simple LED blink:

```
void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

This program might look almost trivially simple, but it demonstrates every core concept of Arduino programming: configuring a pin (`pinMode`), setting a digital output (`digitalWrite`), and timing with `delay`. The same building blocks appear in every more complex program you will ever write on an Arduino — they just get combined in more sophisticated ways.

> ***Workshop:*** The robotic arm in this workshop is controlled by an Arduino Nano

communicating with a PCA9685 over I2C. You will write and upload programs in the Arduino IDE during the hands-on session.

5 Hands-On Guide: Building Your Robotic Arm

You have now built a solid theoretical foundation: you understand what robots are, how they are classified, how their mechanical structures create motion, and how their electrical and control systems bring them to life. Now it is time to put that knowledge to work.

In this hands-on guide, you will go through the complete process of building a working robotic arm — step by step. You will assemble the mechanical structure, wire up the electronics, write the programs that bring it to life, and finally control it from a computer or a smartphone. Every step connects directly to something you have already studied. When you tighten a screw, you are thinking about fasteners. When you plug in a servo, you are thinking about actuators and PWM. When you upload a program, you are giving your robot its instructions.

Take your time, work carefully, and remember: every professional robot engineer started by building something small.

5.1 Safety Guidelines

Before you pick up any components, read through these guidelines carefully. Following them will protect both you and your equipment throughout the session.

- **Never connect or disconnect wires while the power supply is switched on.** Always power down first, make your changes, check your connections, and then power back on.
- **Do not overtighten screws on 3D-printed parts.** Plastic threads strip easily. Tighten until the part is firm and does not wobble, then stop.
- **Always check wire polarity before connecting power.** Reversing positive

and negative connections can damage or destroy components permanently.
- **Keep your workspace tidy.** Loose wires and stray screws can accidentally short-circuit electronics or cause trips and falls.
- **If anything smells like burning, disconnect power immediately** and inform your instructor.
- **Handle servo motors gently.** Do not force them past their mechanical limits — if you hear or feel the gears grinding, stop immediately and reduce the commanded angle.

5.2 Phase 1: Mechanical Assembly

The mechanical assembly transforms a collection of 3D-printed plastic parts, servo motors, and small screws into the physical body of the robotic arm. Follow each step in order and refer to the accompanying figures. Use a screwdriver that fits the screw head correctly — firm tightening is enough, do not use excessive force.

5.2.1 Step 1: Base Assembly

Required Components:
- Base Cover Box — 1 pc.
- Base Mount — 1 pc.
- Servo Motor — 1 pc.
- M2×10 Screws — 4 pcs.
- M2×6 Screw — 1 pc.

Place the **Base Mount** on top of the **Base Cover Box** and align the four screw holes around the perimeter. These two parts form the foundation of the entire arm — the fixed base on which all other links will pivot. Secure them together using four **M2×10 screws**, one at each corner.

Insert the first **Servo Motor** into the designated slot in the Base Mount, with the servo shaft facing inward toward the holder. This is the base joint — the one that rotates the entire arm horizontally. Route the servo cable through the wire channel to keep it organised.

Secure the motor in place using one **M2×6 screw**.

See Figure 34 for the complete assembly of the base structure.

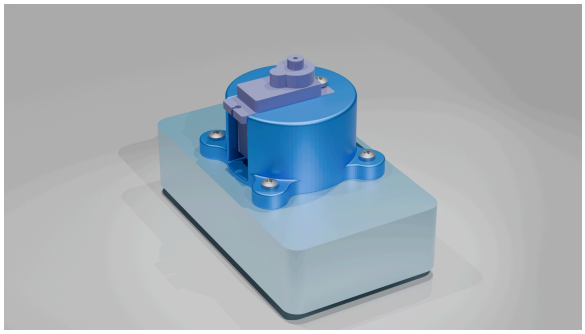


Figure 34: Mechanical Assembly — Step 1: Base Structure.

5.2.2 Step 2: Mounting the First Link

Required Components:
- Link 1 — 1 pc.
- Servo Arm Full — 1 pc.
- Servo Motor — 1 pc.
- M2×6 Screws — 2 pcs.
- M2×3 Screws — 1 pcs.

Place the **Servo Arm Full** into its dedicated groove inside **Link 1**. The servo arm is the mechanical bridge between the rotating shaft of the base motor and the first link — it is what transmits the motor's rotation into the link's movement. Position **Link 1** onto the base structure so that the servo arm aligns correctly with the shaft of the base servo motor. Secure **Link 1** to the servo shaft using one **M2×3 screw**.

Insert the second **Servo Motor** into the motor slot on **Link 1** — this motor will drive **Link 2** in the next step. Secure it using two **M2×6 screws**.

See Figure 35 for the complete assembly of the first link.

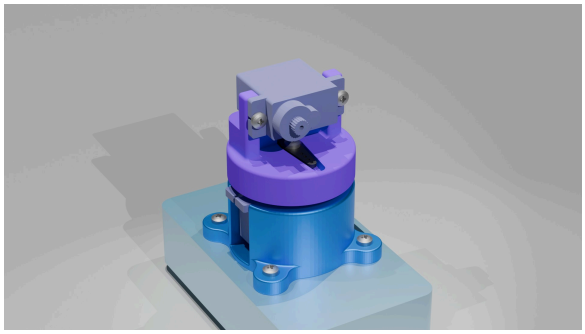


Figure 35: Mechanical Assembly — Step 2: First Link.

5.2.3 Step 3: Mounting the Second Link

Required Components:

- Link 2 — 1 pc.
- Servo Arm Half — 1 pc.
- Servo Motor — 1 pc.
- M2×6 Screws — 2 pcs.
- M2×3 Screw — 1 pc.

Place the **Servo Arm Half** into the groove inside **Link 2**. Align **Link 2** with the shaft of the servo motor mounted on **Link 1** and secure the link using one **M2×3 screw**. Insert the third **Servo Motor** into the motor slot on **Link 2** and fix it in place using two **M2×6 screws**.

See Figure 36 for the complete assembly of the second link.

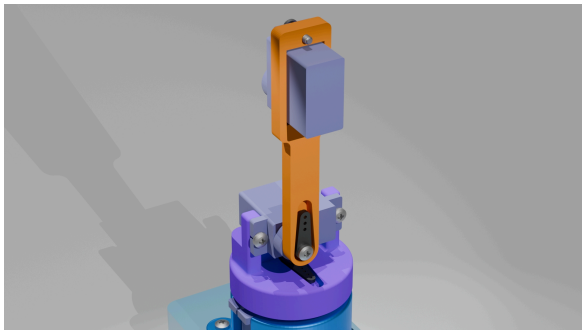


Figure 36: Mechanical Assembly — Step 3: Second Link.

5.2.4 Step 4: Mounting the Third Link

Required Components:

- Link 3 — 1 pc.
- Servo Arm Half — 1 pc.
- Servo Motor — 1 pc.
- M2×6 Screws — 2 pcs.
- M2×3 Screw — 1 pc.

Place the **Servo Arm Half** into the groove in **Link 3**. Align **Link 3** with the shaft of the servo motor on **Link 2** and secure it using one **M2×3 screw**. Mount the fourth **Servo Motor** into the motor slot on **Link 3**. Place the **End Effector Base** on top of servo motor. Secure the servo motor and the end-effector base altogether using two **M2×10 screws**. This motor will drive the end-effector assembled in the next step.

See Figure 37 for the complete assembly of the third link.

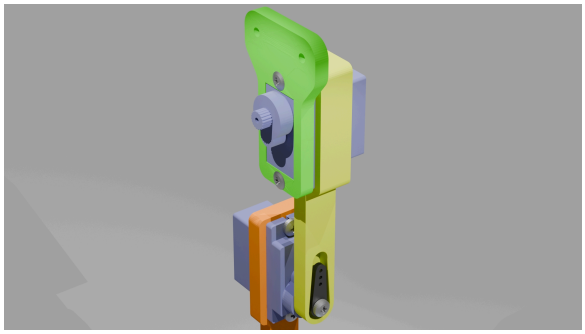


Figure 37: Mechanical Assembly — Step 4: Third Link.

5.2.5 Step 5: Assembling the End-Effector

Required Components:

- Driver Gear — 1 pc.
- Driven Gear — 1 pc.
- Idler Compound Gear — 1 pc.
- End-Effector Claws — 2 pcs.
- Servo Arm Half — 1 pc.
- Servo Motor — 1 pc.
- M2×6 Screw — 1 pc.
- M2×10 Screws — 2 pcs.

This step brings the gear mechanism you studied in the mechanical systems chapter to life. The driver gear, driven gear, and idler compound gear work together to convert the rotation of the servo motor into the opening and closing motion of the two claws. Assemble the gears and end-effector claws as shown in the reference figure, ensuring the gear teeth mesh cleanly without binding or catching.

Mount the **Servo Arm Half** on to the driver gear and then attach this with the servo motor in the Link 3 and secure it with one **M2×3 screw**. Now attache the **Idler Gear** with the right claw using the dedicated grooves and secure it with one **M2×10 screw**. Finally, place the left claw on the left side of the idler gear and secure it with one **M2×10 screw**.

See Figure 38 for the complete assembly of the end-effector.

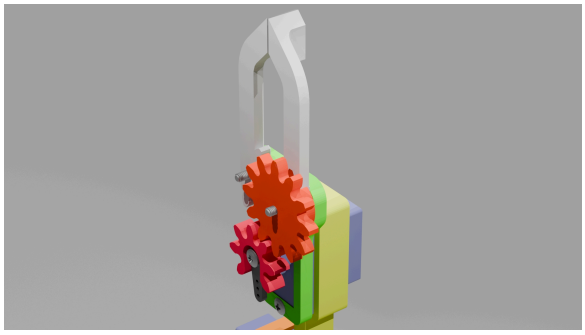


Figure 38: Mechanical Assembly — Step 5: End-Effector with Gear Mechanism.

5.2.6 Final Check: Mechanical Assembly

With all five steps complete, the mechanical structure of the robotic arm is fully assembled. Before moving on, run through this checklist:

- All joints move freely through their range of motion without binding or catching.
- All screws are firm and no parts wobble.
- No servo cables are pinched, kinked, or pulled tight against any moving part.
- The gear teeth in the end-effector mesh cleanly and the claws open and close smoothly by hand.

Figure 39: Complete Mechanical Assembly of the Robotic Arm.

Take a moment to appreciate what you have just built: a three-degree-of-freedom serial manipulator with revolute joints — the same fundamental architecture used in robotic arms across industry worldwide. The base joint rotates the arm horizontally; Links 1, 2, and 3 extend and position the arm in a vertical plane; and the end-effector grips objects. Every concept from the theory chapters is now physical, right in front of you.

5.3 Phase 2: Electrical Connections

With the mechanical structure complete, it is time to bring the electronics together. You will connect four servo motors to the PCA9685 driver, connect the PCA9685 to the Arduino Nano over I2C, and connect an external power supply to run the motors safely.

Work methodically and double-check every connection before applying power.

5.3.1 Understanding the Control Architecture

The control system works in two layers. The **Arduino Nano** is the high-level controller: it receives commands, decides what angle each joint should move to, and passes those instructions to the **PCA9685 servo driver** over I2C. The **PCA9685** handles the low-level work: it independently generates four precise PWM signals and delivers them continuously to the four servo motors. This clean separation — the Nano thinks, the PCA9685 signals, the servos move — is exactly the architecture you studied in the electrical systems chapter.

The four servo channels are assigned as follows:

- Channel 0 → Base Joint
- Channel 1 → Link 1 Joint
- Channel 2 → Link 2 Joint
- Channel 3 → End-Effector Joint

Keep this channel assignment in mind throughout wiring and programming. It is the map between software and hardware.

5.3.2 Connecting Servo Motors to the PCA9685

Each servo motor has a 3-wire cable terminating in a 3-pin connector. The standard colour coding is:

- **Brown or Black** → Ground (GND)
- **Red** → Power (+5 V)
- **Orange or Yellow** → PWM Signal

The PCA9685 output headers present the same three connections — GND, V+, and Signal — from left to right when the chip faces up. Match colours carefully: brown or black to GND, red to V+, orange or yellow to Signal.

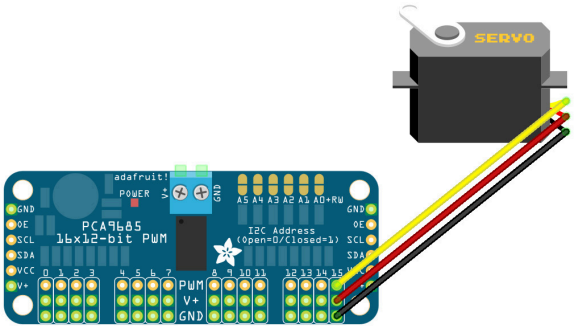


Figure 40: Connecting a servo motor to a channel of the PCA9685 module.

Connect the four servos to channels 0, 1, 2, and 3 in order, following the joint assignment above. If any servo jitters or fails to respond after power-up, switch off immediately and recheck the connector orientation on that channel.

5.3.3 Connecting the PCA9685 to the Arduino Nano

The PCA9685 communicates with the Arduino over I2C using just two signal wires plus power and ground. Make the following four connections:

- **PCA9685 SDA → Arduino A4**
- **PCA9685 SCL → Arduino A5**
- **PCA9685 VCC → Arduino 5V**
- **PCA9685 GND → Arduino GND**

SDA and SCL are the I2C data and clock lines. VCC and GND supply the PCA9685's logic circuitry from the Arduino's regulated 5 V rail. The A4 and A5 pins are the dedicated hardware I2C pins on the ATmega328P — no other pins on the Nano will work for this purpose.

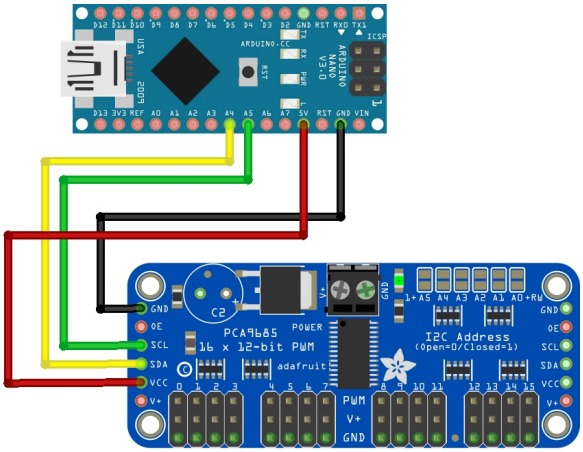


Figure 41: Arduino Nano connected to the PCA9685 module over I2C.

5.3.4 Connecting the External Power Supply

Servo motors draw far more current than the Arduino can safely supply — especially when several joints move at the same time. The servo motors must therefore be powered by a separate **5 V, 3 A DC power supply**, connected directly to the PCA9685's power terminals.

Connect the **positive** output of the power supply to the **V+** terminal of the PCA9685, and the **negative** output to the **GND** terminal. This GND is shared with the Arduino GND through the connections made above, which is essential for the signals to work correctly.

Do not connect the power supply's positive terminal to the Arduino 5 V pin. Always verify polarity before switching on — reversing positive and negative will damage the PCA9685 immediately and permanently.

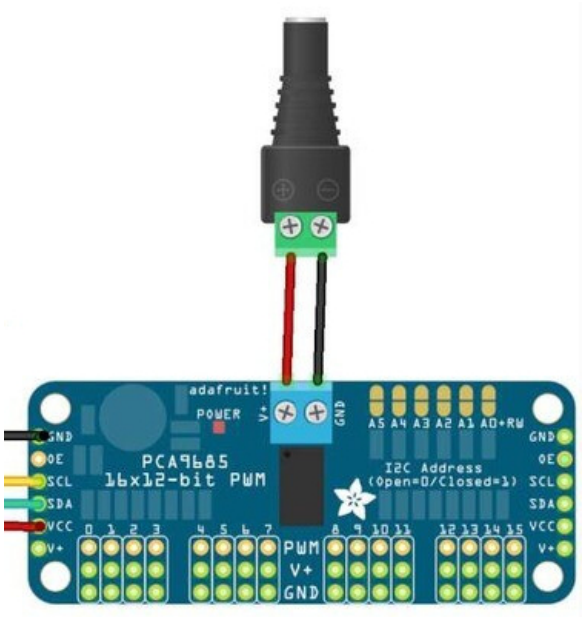


Figure 42: External 5 V power supply connected to the V+ and GND terminals of the PCA9685.

5.3.5 Initial Power-Up Check

With all connections made, verify everything once more visually before applying power. Then connect the Arduino Nano to your computer using its USB cable. The Arduino powers up from USB, and the logic section of the PCA9685 activates through the VCC line.

If wiring is correct, the **red power LED on the PCA9685** will illuminate. This confirms the Arduino is powered, the PCA9685 logic section is active, and the I2C wiring is intact. The servo motors remain unpowered and stationary until you switch on the external 5 V supply — which you should leave off until a program is uploaded and ready.

5.4 Phase 3: Programming and Testing

The hardware is assembled and wired. Now you will set up your programming environment, upload programs to the Arduino Nano, and bring the arm to life.

5.4.1 Setting Up the Arduino IDE

Before uploading any programs, make sure the Arduino IDE is installed and configured correctly on your device.

Step 1 — Download and Install the Arduino IDE

The Arduino IDE 2 is free and available for all platforms. Open a browser, go to *arduino.cc/en/software*, and download for your platform:

Windows — download the `.exe` installer. Run it and accept the USB driver installation prompt — this driver is what allows Windows to recognise the Arduino Nano over USB.

macOS — download the `.dmg` disk image. Open it and drag the Arduino IDE into your Applications folder.

Linux — download the `.AppImage` file. Mark it as executable with `chmod +x` in a terminal and run it directly. No installation step is needed.

Android — open the Google Play Store, search for **Arduino IDE** by **Arduino SA**, and tap Install. Android 8.0 or higher is required, along with a USB OTG adapter to connect the Nano to your device.

Once installed, launch the IDE. On first launch it may take a moment to initialise — this is normal.

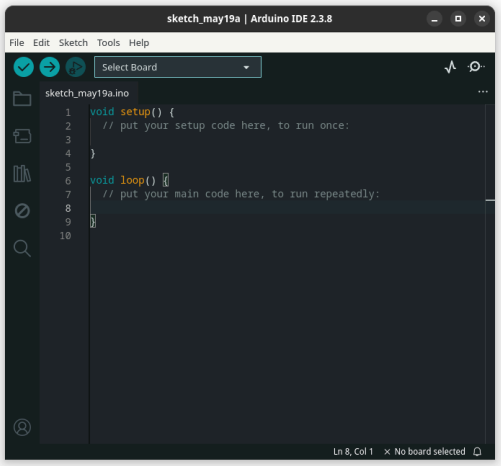


Figure 43: Arduino IDE main interface after first launch on desktop.

Step 2 — Connect the Arduino Nano and Select the Board

1. Connect the **Arduino Nano** to your device via USB. On Android, use a **USB OTG adapter** between your phone and the Nano's cable.
2. Open the board selector — **Tools → Board** on desktop, or the board dropdown at the top of the screen on Android.
3. Select **Arduino AVR Boards → Arduino Nano**.
4. Set the **Processor** to **ATmega328P**. If your Nano uses the CH340 USB chip (common on clone boards), select **ATmega328P (Old Bootloader)** instead.
5. Select the correct port:
   - **Windows:** a COM port such as `COM3`, visible under **Tools → Port**
   - **macOS:** something like `/dev/cu.usbserial-XXXX`, visible under **Tools → Port**
   - **Linux:** typically `/dev/ttyUSB0` or `/dev/ttyACM0`, visible under **Tools → Port**
   - **Android:** the IDE detects the board automatically via OTG

If you are unsure which port is the Arduino on desktop, unplug the USB cable, note what

is listed, plug it back in — the new entry is your board.

On Linux, if uploading fails with a permissions error, run the following in a terminal and log out and back in:

```
sudo usermod -a -G dialout $USER
```

On Windows, if the Nano does not appear under any COM port, the CH340 driver is missing. Search online for **CH340 driver** and install it from the manufacturer's site.

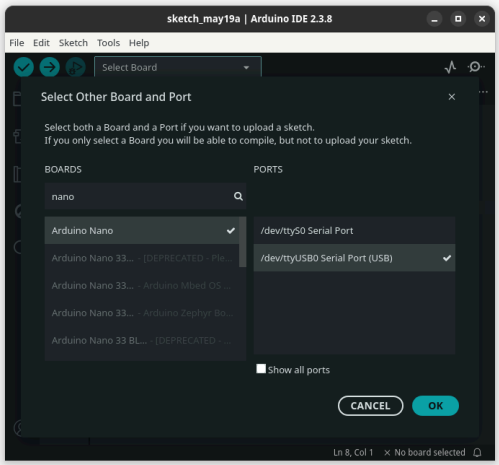


Figure 44: Selecting Arduino Nano and processor in the Tools menu on Linux.

Step 3 — Update Built-In Libraries

Open the Library Manager (**Tools → Manage Libraries** on desktop; hamburger menu → **Library Manager** on Android). Search for **Wire** and click or tap **Update** if available. Do the same for **Servo**. These libraries ship with the IDE but keeping them current ensures the best compatibility.

Step 4 — Install the Required Libraries

This project needs one library that does not come with the IDE:

Adafruit PWM Servo Driver Library (by Adafruit) — communicates with the PCA9685 servo driver over I2C. Its dependency, **Adafruit BusIO**, is installed automatically alongside it.

To install on any platform:

1. Open the Library Manager.
2. Search for **Adafruit PWM Servo Driver**.
3. Confirm the author is **Adafruit**, then click or tap **Install**.
4. When asked about missing dependencies, always choose **Install All**.

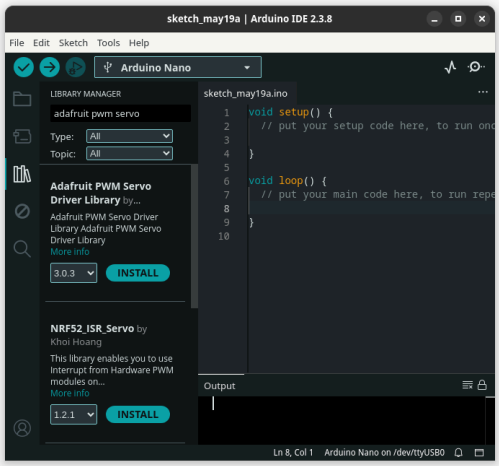


Figure 45: Searching for the Adafruit PWM Servo Driver Library.

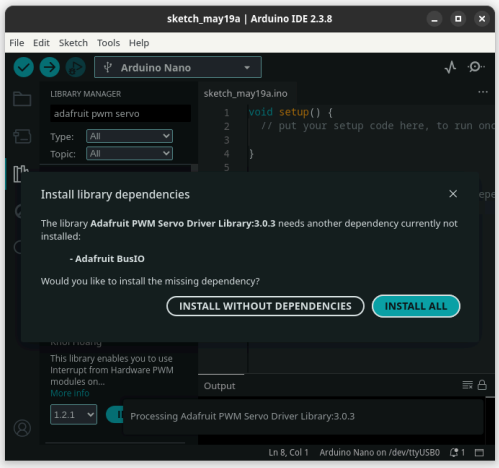


Figure 46: Choosing Install All to include all dependencies.

Installing any other library in the future follows the exact same process. To update existing libraries, open the Library Manager at any time, find entries with an **Update** button, and click or tap it.

Step 5 — Verify with a Blank Upload

Open a new blank sketch (**File → New Sketch** on desktop; tap + on Android), leave it empty, and click or tap **Upload**. The status bar should read **Done uploading** with no errors. This confirms the board, port, and driver are all correctly configured before you start writing robot code.

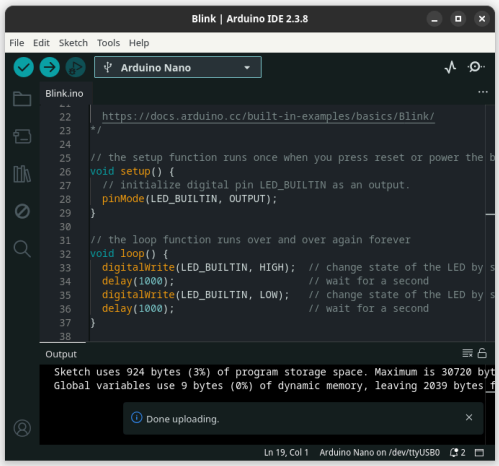


Figure 47: Done uploading confirmation in the Arduino IDE.

If the upload fails, check: correct board and processor selected? Correct port selected? CH340 driver installed (Windows)? Added to `dialout` group (Linux)? USB permission granted (Android)?

5.4.2 Serial Monitor Tools

Once a program is running on the Arduino, a **serial monitor** is how you communicate with it — sending commands and reading responses over the USB connection. The Arduino IDE has a built-in Serial Monitor (the magnifying glass icon, top-right) that is perfectly capable for this project. Set the baud rate to **115200** to match the robot controller program, type a command in the input box, and press Enter to send.

If you prefer a standalone tool with more features such as data logging or live plotting, many excellent options exist. On **Windows**, QtSerialMonitor and PuTTY are popular choices. On **Linux**, Serial Studio and Minicom are widely used. On **macOS**, CoolTerm and the built-in `screen` command work well. On **Android**, Serial USB Terminal by Kai Morich connects over USB OTG. Every tool works the same way at its core — select the port, set the baud rate to 115200, and start communicating. You are encouraged to explore whichever tool suits your workflow best.

5.4.3 Testing All Servo Motors

Before running the full robot controller, always start with a simple test that verifies every motor is working correctly. This catches wiring errors, mechanical binding, and software issues early — when they are easiest to fix.

The following program moves all four servo motors between two positions repeatedly. Upload it to the Arduino Nano:

```
/*
 * File: robot_test_example1.ino
 *
 * SPDX-FileCopyrightText: 2026 Dhiman Sarkar, National Council of Science Museums (NCSM)
 * SPDX-License-Identifier: AGPL-3.0-or-later
 */

#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>

// Arduino Nano I2C Pins:
// SDA -> A4
// SCL -> A5

Adafruit_PWMServoDriver pca9685 = Adafruit_PWMServoDriver();

#define SERVOMIN  102
#define SERVOMAX  512

void setup() {
  Wire.begin();

  pca9685.begin();
  pca9685.setPWMFreq(50);

  delay(500);
}

void loop() {

  // Move servos from 30° to 60°
  for (int angle = 30; angle <= 60; angle++) {
    int pulse = map(angle, 0, 180, SERVOMIN, SERVOMAX);

    // setPWM(channel, ON_time, OFF_time)
    // channel  -> PCA9685 output channel number
    // ON_time  -> PWM start count (usually 0)
```

```
    // OFF_time -> PWM end count (controls servo position)

    pca9685.setPWM(0, 0, pulse);
    pca9685.setPWM(1, 0, pulse);
    pca9685.setPWM(2, 0, pulse);
    pca9685.setPWM(3, 0, pulse);

    delay(20);
  }

  delay(1000);

  // Move servos from 60° back to 30°
  for (int angle = 60; angle >= 30; angle--) {
    int pulse = map(angle, 0, 180, SERVOMIN, SERVOMAX);

    pca9685.setPWM(0, 0, pulse);
    pca9685.setPWM(1, 0, pulse);
    pca9685.setPWM(2, 0, pulse);
    pca9685.setPWM(3, 0, pulse);

    delay(20);
  }

  delay(1000);
}
```

After uploading, switch on the external 5 V power supply and observe the arm. All four joints should move smoothly back and forth. If any servo does not respond, switch off power immediately and recheck its wiring to the PCA9685 channel. If a joint moves in the wrong direction or binds, note the channel number and investigate the assembly of that joint.

A successful test confirms that the Arduino is running, I2C communication to the PCA9685 is working, and all four servo motors are operational. You are ready to move on.

5.4.4 Understanding Joint Angles and PWM Values

Each servo motor position is controlled by a PWM count value sent to the PCA9685. The PCA9685 uses 12-bit resolution — values from 0 to 4095 — and the pulse width each count produces depends on the PWM frequency, which is set to 50 Hz.

For standard hobby servo motors at 50 Hz:

- A PWM count of approximately **102** corresponds to 0°
- A PWM count of approximately **512** corresponds to 180°

The exact mapping varies slightly between servo models. It is good practice to verify the minimum and maximum PWM counts for each servo experimentally before running complex movements. If a servo grinds — its gears are being driven past the mechanical stop — reduce the PWM value immediately.

5.4.5 The Robot API: Controlling the Arm Through Serial Communication

At this point it is worth introducing an important concept that will change how you think about the robotic arm as a system.

An **API** — short for **Application Programming Interface** — is a defined, agreed-upon way for two systems to communicate and exchange information. You encounter APIs constantly in the technology world: a weather app on your phone uses an API to request data from a weather server; a payment system uses an API to communicate with a bank. The key idea is that an API **hides the complexity underneath** and exposes only a clean, simple interface. You do not need to know how the bank's internal servers work to make a payment — you just use the API.

Your robotic arm has an API too. It is the **serial data format** the Arduino controller listens for:

`ang1,ang2,ang3,ang4`

This is a simple but complete hardware API. Any device or program that can send text over a serial connection — a computer, a smartphone, a Raspberry Pi, another microcontroller, or a custom application — can control the robotic arm fully by speaking this format. The sender does not need to know anything about PWM signals, the PCA9685 driver, servo wiring, or the ATmega328P microcontroller. All of that complexity is hidden behind the Arduino controller. The API is the clean surface that everything else talks to.

This is exactly how real industrial robots work. A robot arm in a factory does not

expose its individual motor controllers to the factory management software. Instead, it exposes a defined command interface — an API — and the software above simply sends movement instructions. The robot handles everything underneath.

The communication parameters for this arm's API are:

- **Interface:** Serial over USB (UART)
- **Baud Rate:** 115200
- **Command format:** `ang1,ang2,ang3,ang4` — four comma-separated decimal values in degrees, no spaces
- **Success response:** none (the arm moves silently)
- **Error response:** `ERROR` — sent back when a malformed or incomplete command is received

Example valid command: `45.0,90.0,120.0,60.0`

> ***Workshop:*** This serial API is the communication layer the workshop's Python web controller uses to drive the arm. Any program or device you write in the future can control this arm using the same simple format — without touching the Arduino code at all.

5.4.6 Running the Robot Controller Program

Upload the following program to the Arduino Nano. This is the main controller — it listens continuously for incoming serial commands in the API format described above and moves the arm joints accordingly:

```
/*
 * File: robot_test_example1.ino
 *
 * SPDX-FileCopyrightText: 2026 Dhiman Sarkar, National Council of Science Museums (NCSM)
 *
 * SPDX-License-Identifier: AGPL-3.0-or-later
*/

#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>

Adafruit_PWMServoDriver pca9685 = Adafruit_PWMServoDriver();

int servoMin = 102;
int servoMax = 512;

void setup() {
  Serial.begin(115200);

  Wire.begin();

  pca9685.begin();
  pca9685.setPWMFreq(50);
  pca9685.setOscillatorFrequency(25000000);
}

void loop() {
  if(Serial.available()) {
    String data = Serial.readStringUntil('\n');

    data.trim();

    // Check comma positions
    int c1 = data.indexOf(',');
    int c2 = data.indexOf(',', c1 + 1);
    int c3 = data.indexOf(',', c2 + 1);

    if(c1 == -1 || c2 == -1 || c3 == -1) {
      Serial.println("ERROR");
      return;
    }

    // Extract values
    String s1 = data.substring(0, c1);
    String s2 = data.substring(c1 + 1, c2);
    String s3 = data.substring(c2 + 1, c3);
    String s4 = data.substring(c3 + 1);

    // Convert to angles
    float ang1 = s1.toFloat();
    float ang2 = s2.toFloat();
    float ang3 = s3.toFloat();
    float ang4 = s4.toFloat();

    // Validate range
    if(
      ang1 < 0 || ang1 > 180 ||
      ang2 < 0 || ang2 > 180 ||
      ang3 < 0 || ang3 > 180 ||
      ang4 < 0 || ang4 > 180
    ) {
      Serial.println("ERROR");
      return;
    }

    // Convert angle to PWM
    int pwm1 = map(ang1, 0, 180, servoMin, servoMax);
    int pwm2 = map(ang2, 0, 180, servoMin, servoMax);
    int pwm3 = map(ang3, 0, 180, servoMin, servoMax);
    int pwm4 = map(ang4, 0, 180, servoMin, servoMax);

    // Update servos
    // setPWM(channel, start, end)
    pca9685.setPWM(0, 0, pwm1);
    pca9685.setPWM(1, 0, pwm2);
    pca9685.setPWM(2, 0, pwm3);
    pca9685.setPWM(3, 0, pwm4);

    Serial.println("OK");
  }
}
```

When a valid command arrives, the program extracts the four angle values, converts each into the corresponding PWM count using a

linear mapping (0° → minimum count, 180° → maximum count), and sends all four updated values to the PCA9685, causing the joints to move to their new positions simultaneously.

If an incomplete or incorrectly formatted command arrives, the controller ignores it entirely and responds with `ERROR` — protecting the arm from unexpected motion caused by corrupted data.

You can test the controller immediately using the Serial Monitor in the Arduino IDE. With the 5 V supply on and the program running, type a command such as `45.0,90.0,60.0,30.0` into the Serial Monitor input box and press Enter. The arm should respond and move its joints to those angles.

5.4.7 Controlling the Arm from a Web Interface

Alongside this document, a **Python controller script** is supplied that provides a browser-based graphical interface for controlling the robotic arm. Instead of typing raw serial commands into a terminal, you interact with a visual control panel in any web browser — sliders, buttons, and real-time joint angle readouts — while the Python script translates your inputs into the same serial API commands the Arduino expects.

This is a direct demonstration of what the API makes possible. The Python script does not know or care about servo motors, PWM signals, or the PCA9685. It only speaks the API — four comma-separated angles over serial — and the Arduino handles everything from there. The same Python script would work equally well if you rebuilt the arm with completely different motors and a different driver board, as long as the Arduino firmware behind the API remained the same.

To run the Python controller:

1. Ensure **Python 3** is installed on your computer. Download it from *python.org* if needed.
2. Install the required Python libraries by running the following in a terminal or command prompt:

```
pip install pyserial flask
```

1. Connect the Arduino Nano (running the controller program from the previous step) to your computer via USB and switch on the 5 V power supply.
2. Open a terminal, navigate to the folder containing the supplied Python script, and run:

```
python robot_controller.py
```

1. The script will print a local web address — typically `http://127.0.0.1:5000` or `http://localhost:5000` — to the terminal. Open this address in any web browser on the same computer.
2. The control interface will load. Use the sliders to set the desired angle for each joint and click the Send or Move button. The Python script formats your input as a serial API command and sends it to the Arduino, which moves the arm.

If the script cannot find the Arduino's serial port, open the Python file in a text editor and update the port name at the top of the file to match the COM port (Windows) or `/dev/` path (Linux/macOS) that the Arduino Nano is connected to.

5.4.8 Wireless Control from Android

If you would like to control the arm without a USB cable, you can add a **Bluetooth serial module** — such as the widely available HC-05 or HC-06 — to the Arduino Nano. These modules appear to the Arduino as a standard UART serial connection, so the robot controller program requires no changes whatsoever. The arm simply receives the same serial API commands wirelessly.

Wiring the HC-05 or HC-06 to the Arduino Nano:

- HC-05/06 **VCC** → Arduino **5V**
- HC-05/06 **GND** → Arduino **GND**

- HC-05/06 **TXD** → Arduino **RX (Pin 0)**
- HC-05/06 **RXD** → Arduino **TX (Pin 1)**

Important: disconnect the Bluetooth module's TX and RX wires whenever uploading a new sketch, as the serial port is shared and the module will interfere with the upload.

To pair and connect from Android, install **Serial Bluetooth Terminal** by Kai Morich from the Google Play Store. Power on the Arduino with the module connected — the module's LED will blink rapidly. On your Android device, open **Settings → Bluetooth**, scan for devices, and tap the HC-05 or HC-06 entry. Enter the pairing PIN: **1234** for HC-06, or **0000** for HC-05. Once paired, open Serial Bluetooth Terminal, tap the connect icon, select your module, and set the baud rate to **115200**. Send commands in the same API format — `45.0,90.0,120.0,60.0` — and the arm will respond exactly as it does over USB.

5.5 Final Notes

Congratulations — your robotic arm is fully assembled, wired, programmed, and under your control.

You have gone from a collection of parts to a working, computer-controlled robotic system. Along the way you have practised mechanical assembly, electronics wiring, embedded programming, serial communication, and the concept of a hardware API. Each of those skills appears in real-world robotics and engineering at every level of complexity.

From here, you can go in many directions. Experiment with different joint angle combinations to explore the arm's workspace and understand its limits. Modify the Python web controller to add new features or a different interface. Write your own program — in any language — that speaks the serial API and drives the arm in a pattern of your design. Add sensors to the Arduino and feed their readings back through the serial connection to make the arm react to the world around it.

The architecture you have built — serial manipulator, servo actuators, PWM driver, microcontroller controller, serial API — is the same foundation used in desktop robotic arms, surgical systems, CNC machines, and research platforms worldwide. You now understand it from the inside out. The next step is entirely yours.

Bibliography

[1] R. N. Jazar, *Theory of Applied Robotics: Kinematics, Dynamics, and Control*, 3rd ed. Springer, 2022. doi: 10.1007/978-3-030-93220-6.

[2] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed. Pearson Prentice Hall, 2005.

[3] S. B. Niku, *Introduction to Robotics: Analysis, Control, Applications*, 3rd ed. Wiley, 2020.

[4] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017. [Online]. Available: http://hades.mech.northwestern.edu/index.php/Modern_Robotics

[5] R. M. Murray, Z. Li, and S. S. Sastry, *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994. [Online]. Available: https://www.cds.caltech.edu/~murray/mlswiki

[6] P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*, 2nd ed. Springer, 2017. doi: 10.1007/978-3-319-54413-7.

[7] Arduino, "Arduino Language Reference." [Online]. Available: https://www.arduino.cc/reference/en/

[8] Adafruit Industries, "Adafruit PCA9685 16-Channel PWM Servo Driver: Overview and Wiring Guide." [Online]. Available: https://learn.adafruit.com/16-channel-pwm-servo-driver

[9] WRO India, "World Robot Olympiad India — Season 2026 Category Rules." [Online]. Available: https://wroindia.org/category-rules-2026/

[10] National Council of Science Museums, "World Robot Olympiad 2026 — NCSM Support Programme." [Online]. Available: https://ncsm.gov.in/activities-main/events/world-robot-olympiad-2026

[11] J. Denavit and R. S. Hartenberg, "A Kinematic Notation for Lower-Pair Mechanisms Based on Matrices," *Journal of Applied Mechanics*, vol. 22, pp. 215–221, 1955.

[12] D. E. Whitney, "Resolved Motion Rate Control of Manipulators and Human Prostheses," *IEEE Transactions on Man-Machine Systems*, vol. 10, no. 2, pp. 47–53, 1969, doi: 10.1109/TMMS.1969.299896.

[13] J. F. Engelberger, "Robotics in Practice: Management and Applications of Industrial Robots," 1980.

[14] D. Rus and M. T. Tolley, "Design, Fabrication and Control of Soft Robots," *Nature*, vol. 521, pp. 467–475, 2015, doi: 10.1038/nature14543.

[15] V. Mnih, K. Kavukcuoglu, D. Silver, and others, "Human-Level Control Through Deep Reinforcement Learning," *Nature*, vol. 518, pp. 529–533, 2015, doi: 10.1038/nature14236.

List of Figures

Robotics Workshop eResources

🚀 Learn More • Practice More • Build More 🤖

📚 Available Resources

Inside the repository, students can find:

- 🤖 Robotic Arm project files
- 💻 Example codes and programming files
- 🛠️ 3D printing models and design files
- 📖 Workshop notes and learning materials
- 🎞️ Presentation slides (PPTs)
- 📦 Updated instructions and guides
- 🚀 Extra references and practice resources

🌐 Official Workshop Repository

Scan the QR Code

or visit the repository online:

🔗 github.com/DhimanSarkar-NCSM-Exhibits/Robotic-Arm

🧭 How to Use

1. Open the GitHub repository link
2. Browse the folders and workshop resources
3. Download files or clone the repository
4. Practice and experiment on your own
5. Visit regularly for updates and new materials

Keep Exploring the World of Robotics!

Your next invention could start with a single idea. 🚀